

**Design and Implementation of a Driver Drowsiness Detection and Speed Control
System for Truck Drivers.**

BY

**ASHIMOLOWO OLUWASEGUN
(20CK027511)**

**A PROJECT REPORT SUBMITTED TO THE DEPARTMENT OF
ELECTRICAL & INFORMATION ENGINEERING, IN PARTIAL
FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF THE
BACHELOR OF ENGINEERING DEGREE (B.Eng) IN ELECTRICAL AND
ELECTRONICS ENGINEERING, COVENANT UNIVERSITY, OTA, OGUN
STATE, NIGERIA.**

**SUPERVISOR(S):
DR VICTORIA OGUNTOSIN**



JUNE, 2025

DECLARATION

I hereby declare that I carried out the work reported in this project in the Department of Electrical and Information Engineering, Covenant University, under the supervision of Dr. Victoria Oguntosin. I also solemnly declare that to the best of my knowledge; no part of this report has been submitted here or elsewhere in a previous application for the award of a degree. All sources of knowledge used have been duly acknowledged.

 18/06/25

ASHIMOLOWO OLUWASEGUN

20CK027511

CERTIFICATION

This is to certify that the project titled "Design and Implementation of a Driver Drowsiness Detection and Speed Control System for Truck Drivers." by ASHIMOLOWO OLUWASEGUN, meets the requirements and regulations governing the award of the Bachelor of Engineering (Electrical and Electronics Engineering) degree of Covenant University and is approved for its contribution to knowledge and literary presentation.

Supervisor 1: Sign: VO 18/06/2025

Name: Dr. Victoria Oguntosin Date: 18/06/2025

Internal Examiner: Sign: _____

Name: _____ Date: 18/06/2025

Head of Department: Sign: _____

Name: Dr. Isaac A. Samuel Date: 18/06/2025

DEDICATION

I would like to dedicate this project to God for his guidance and direction for the successful completion of this project. I would also like to dedicate this project to my family for their provision and unconditional support over the years. I appreciate the university for this opportunity and my supervisor for his steadfast support and knowledge.

ACKNOWLEDGEMENT

My gratitude goes to the one and only God Almighty, King of Kings and Lord of Lords who provided me guidance throughout the course of this work. I would like to acknowledge my supervisor, Dr. Victoria Oguntosin for helping me create a project that could significantly impact society. I would like to acknowledge her positive push on me to go beyond the norms and mediocre standards. I would also like to acknowledge the Electrical and Information Engineering department, faculty and staff of Covenant University that have contributed to my learning process within the years I spent in school.

TABLE OF CONTENTS

DECLARATION	2
CERTIFICATION	3
DEDICATION.....	4
ACKNOWLEDGEMENT	5
TABLE OF CONTENTS	6
LIST OF FIGURES	9
LIST OF TABLES	10
LIST OF ABBREVIATIONS AND SYMBOLS	11
ABSTRACT	13
CHAPTER 1.....	14
INTRODUCTION	14
1.1 Background of the Study	14
1.2 Significance and Motivation of the Study	14
1.3 Problem Statement	15
1.4 Aim and Objectives	15
1.5 Methodology.....	16
1.6 Scope and Limitations of the Study.....	17
1.7 Project Report Organization	17
CHAPTER 2.....	19
LITERATURE REVIEW	19
2.1 Introduction.....	19
2.2 Key Indicators of Driver Drowsiness.....	19
2.2.1 Subjective Sleepiness Scales	19
2.2.2 Objective Physiological and Behavioral Signs	20
2.3 Drowsiness Detection Modalities.....	21
2.3.1 Physiological Approaches.....	21
2.3.2 Vehicular Approaches	24

2.3.3 Behavioral (Vision-Based) Approaches	25
2.4 Integrated Warning and Speed-Control Frameworks	29
2.4.1 Two-Stage Intervention	30
2.4.2 Vehicle-to-Everything (V2X) Alerts	31
2.4.3 Adaptive Cruise Control (ACC) Enhancement	31
2.5 Comparative Review of Related Works	32
2.6 Challenges and Emerging Directions	34
2.6.1 Sensor Intrusiveness vs. Driver Comfort	34
2.6.2 Lighting Conditions and Environmental Variability	35
2.6.3 Dataset Standardization and Deployment Gaps	35
2.6.4 Hybrid Sensor-Fusion Architectures	35
2.7 Chapter Summary	36
CHAPTER 3	37
SYSTEM ANALYSIS AND DESIGN	37
3.1 Introduction	37
3.2 System Requirements	37
3.2.1 Raspberry Pi 4	37
3.2.2 Raspberry Pi USB Camera	37
3.2.3 12 V Battery Input Connector	38
3.2.4 USB-A Port	38
3.2.5 Buck Converter (LM2596S-5V)	38
3.2.6 Inductor (47 μ H)	38
3.2.7 Bulk Capacitors (4700 μ F/16 V)	38
3.2.9 Smoothing Capacitor (220 μ F/35 V)	39
3.2.10 Buzzer	39
3.2.11 Indicator LEDs	39
3.2.12 Blocking Diode (1N4007)	39
3.2.13 Schottky Diode (SS34)	39
3.2.14 Transistors (2SC1815)	39

3.2.15 MOSFET Driver (IRF4905)	40
3.2.16 Resistor Networks	40
3.2.17 I2C Led 1602 with Module.	40
3.2.18 GY Max30102	40
3.2.19: Electric TT Magnetic DC Motor Gearbox dual shaft geared Axis	41
3.3 System Design Specification	42
3.3.1 Hardware Design	42
3.3.2 Software Design	45
3.4 System Implementation	47
3.4.1 Hardware Assembly	47
3.4.2 Software Deployment	53
3.4.3 System Testing and Validation	54
3.4 Chapter Summary	54
CHAPTER 4	56
IMPLEMENTATION AND TESTING	56
4.1 Introduction	56
4.2 Testing, Results and Discussions	56
4.3 Bill of Engineering Measurements and Evaluation	61
4.4 Chapter Summary	63
CHAPTER 5	64
CONCLUSION AND RECOMMENDATIONS	64
5.1 Summary	64
5.2 Achievements	64
5.3 Recommendations	65
REFERENCES	66
APPENDIX	69

LIST OF FIGURES

FIGURE 1: BLOCK DIAGRAM OF PROJECT METHODOLOGY	16
FIGURE 2: FRAMEWORK OF PHYSIOLOGICAL PARAMETERS-BASED TECHNIQUES.....	22
FIGURE 3: EEG COMPONENTS AND WORKFLOW	23
FIGURE 4: ARCHITECTURE OF DROWSINESS DETECTION BASED ON EYE STATE	27
FIGURE 5: CNN ARCHITECTURE	28
FIGURE 6: FUZZY INFERENCE SYSTEM	29
FIGURE 7: PICTURES OF COMPONENTS.....	41
FIGURE 8: DESIGN SPECIFICATION.....	42
FIGURE 9: SCHEMATIC DIAGRAM.....	47
FIGURE 10: PCB DESIGN	48
FIGURE 11: COPPER SLAB.....	49
FIGURE 12: GLOSSY PAPER ON COPPER	49
FIGURE 13: ETCHING AND CHEMICAL SOLUTION	50
FIGURE 14: CIRCUIT BOARD TOP VIEW	50
FIGURE 15: PCB BOARD DESIGN	51
FIGURE 16: PCB BOARD	51
FIGURE 17: CIRCUIT CONNECTION.....	51
FIGURE 18: MOUNTING OF RASPBERRY PI	52
FIGURE 19: LCD WITH CAMERA AND SETUP.....	57
FIGURE 20: ASSEMBLED SETUP.....	58
FIGURE 21: SETUP DURING TESTING.....	59
FIGURE 22: LOGIC FOR EYE CLOSED	60
FIGURE 23: LOGIC FOR ABSENT DRIVER.....	60
FIGURE 24: FACE AND EYE DETECTION CLASSIFIERS	60
FIGURE 25: FACE DETECTION	60
FIGURE 26: EYE DETECTION INSIDE EACH FACE	60

LIST OF TABLES

TABLE 1: KAROLINSKA SLEEPINESS SCALE	20
TABLE 2: WIERWILLE AND ELLSWORTH DROWSINESS SCALE.....	20
TABLE 3: REVIEW OF RELATED WORKS	32
TABLE 4: BILL OF ENGINEERING MEASUREMENT AND EVALUATION	62

LIST OF ABBREVIATIONS AND SYMBOLS

AI:	Artificial intelligence
ML:	Machine Language
IoT:	Internet of things
ADAS:	Advanced Driver Assistance Systems
RaDAR:	Radio Detection and Ranging
LiDAR:	Light Detection and Ranging
LDW:	Lane Departure Warning
ACC:	Adaptive Cruise Control
DMS:	Driver Monitoring Systems
V2V:	Vehicle to Vehicle
V2X:	Vehicle to Everything
V2I:	Vehicle to Infrastructure
V2N:	Vehicle to Network
EAR:	Eye Aspect Ratio
LCD:	Liquid Crystal Display
KSS:	Karolinska sleepiness Scale
EEG:	Electroencephalography
ECG:	Electrocardiography
EOG:	Electrooculography
GSR:	Galvanic skin response
SVM:	Support Vector Machines
CNN:	Convolutional Neural Networks
RNN:	Recurrent Neural Networks
HRV:	Heart Rate Variability
PERCLOS:	Percentage of Eyelid Closure over Pupil over Time
ROC:	Receiver Operation Curve

PPG:	Photo Plethysmo Graphy
SWA:	Steering Wheel Angles
CNN	Convolutional Neural network
TCNN	Tweaked Convolutional Neural network
DCNN	Deep Convolutional Neural network
GMM	Gaussian Mixture Model
DAN	Deep Alignment Network

ABSTRACT

Drowsy driving remains a major contributor to road accidents worldwide. According to the National Highway Traffic Safety Administration (NHTSA), over 100,000 crashes annually in the United States are caused by fatigued drivers. This project presents the design and implementation of a Driver Drowsiness Detection and Speed Control System using Raspberry Pi 4 as the central processing unit. The system integrates computer vision-based fatigue detection with physiological signal monitoring using the GY-MAX30102 Pulse Oximeter sensor. Real-time video input from a webcam was processed using OpenCV and Dlib libraries to extract facial features such as eye aspect ratio (EAR), PERCLOS, and mouth aperture for detecting signs of drowsiness. When fatigue indicators exceeded set thresholds, the system triggered visual and auditory alerts via an I²C LCD screen and buzzer. Additionally, a motor speed control mechanism was implemented to regulate vehicle throttle in response to detected fatigue states. The system demonstrated robust performance under indoor lighting conditions, achieving a high accuracy of about 80% in real-time drowsiness detection with minimal latency of less than 100ms inference latency and response time of less than 10ms alert activation following drowsiness threshold breach. Hardware components were integrated efficiently into a compact setup powered by a 12 V battery through a regulated power supply circuit. Testing revealed that the system could effectively detect closed eyes and reduced alertness, triggering appropriate warnings and initiating automated speed reduction when necessary.

Keywords : Driver Drowsiness, Computer Vision, GY-MAX30102, Raspberry Pi, EAR, PERCLOS, Motor Control, IoT, Embedded System.

CHAPTER 1

INTRODUCTION

1.1 Background of the Study

Driver drowsiness is one of the major causes of road accidents worldwide, which contributes to significant damage to life, injuries and economic costs. According to the World Health Organization (WHO), approximately 1.35 million people die annually due to road traffic accidents, fatigue of the driver is a major contribution factor in these accidents 20–30% of these incidents [9]. Drowsiness disrupts a quick decision-making ability, reducing the response time, and increases the possibility of accidents, especially during long-distance driving or night travel.

Traditional methods of detecting drowsiness rely on subjective self-assessment by drivers or observable signs such as irregular driving patterns. However, these methods are often incredible, as drivers cannot recognize their fatigue or ignore warning signs until it is too late. With the advancement of technology, especially Computer Vision, Machine Learning, and in the areas of Internet of Things (IOT), there is an increasing opportunity to develop automated systems that can detect driver's drowsiness in real time and take preventive measures to ensure road safety.

The project focuses on designing and implementing a driver's drowsiness detection system and control which takes advantage of facility analysis, behaviour monitoring, and vehicle-based sensors. Once drowsiness is detected, the system will alert the driver and if necessary, take control of the vehicle to prevent accidents. The purpose of the system is to reduce the number of accidents related to drowsiness by providing an initial warning mechanism and ensuring timely intervention.

1.2 Significance and Motivation of the Study

The importance of this study lies in its ability to save life and reduce the economic burden caused by accidents related to drowsiness. According to National Highway Traffic Safety Administration (NHTSA), estimated 72,000 crash, 44,000 injuries, and 800 deaths in the

United States annually. In developing countries, where road safety rules are less rigid, the problem is even more clear[5].

Inspiration for this project stems from the need to address this important issue using modern technology. By integrating computer vision, machine learning, and IOT, the system can provide a reliable and non-intrusive solution to detect the drowsiness of the driver. The system is designed to be cost-effective, easier to apply, and scalable, which makes it suitable for both individual and commercial vehicles.

1.3 Problem Statement

Despite the progress in vehicle safety technologies, accidents related to drowsiness remain a significant problem. Current solutions, such as lane departure warning system and fatigue detection algorithms, often rely on single indicators (e.g. steering patterns or eyes blink rate) and may not be accurate in all driving conditions. Additionally, these systems are often expensive and are not accessible to the average driver. [4], [9], [12], [17], [25].

The problem of solving this project is a deficiency of a broad, real-time and inexpensive system which can correctly detect the drowsiness of the driver using several indicators (e.g. the closure of the eye, yawning, head position and steering pattern) and takes appropriate action to prevent accidents. The system must be able to operate in various lighting and driving conditions, ensuring reliability and accuracy.

1.4 Aim and Objectives

The primary aim of this project is to design and implement a Driver Drowsiness Detection System and Control that can detect signs of drowsiness in real-time then slow down the motors of the vehicle until it come to a halt while alerting the driver to take preventive measures and ensure road safety.

The objectives of the project are as follows:

1. Design and implement a hardware system that integrates Raspberry pi 3 USB camera with GY max30102 pulse oximeter sensor and microcontrollers to monitor driver behavior and vehicle dynamics.
2. To develop a real-time drowsiness detection system using facial landmarks.

3. To develop a circuit diagram showing the connections between the associated components

4. Train and validate the face and eye model

1.5 Methodology

The flowchart in Figure 1 provides a clear visual representation of the sequential steps involved in achieving the project. The arrows indicate the flow of tasks, and each process is encapsulated within its respective box.

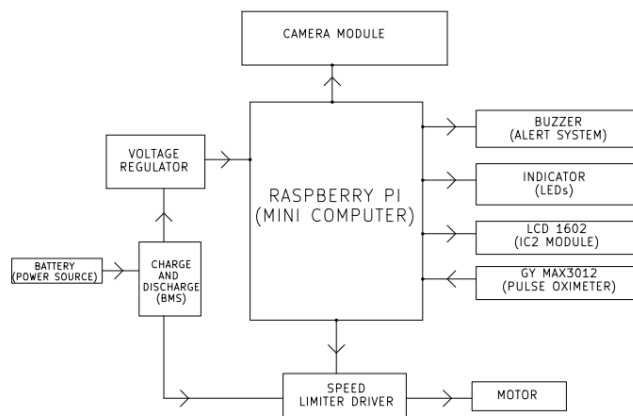


Figure 1: Block Diagram of Project Methodology

The methodology for this project involves a multi-step approach encompassing data acquisition, drowsiness detection, LCD interface development, alert and control systems, and testing. First, a camera is used to capture real-time video of the driver's face, while integrated sensors such as accelerometers and gyroscopes monitor vehicle dynamics like steering patterns and lane departure. Drowsiness detection is carried out using OpenCV and Dlib libraries to identify facial landmarks such as the eyes and mouth. From these inputs, the system calculates key metrics including Eye Aspect Ratio (EAR), Percentage of Eye Closure (PERCLOS), and yawning frequency. Image data undergoes preprocessing steps such as normalization and resizing to 224x224 pixels for model input, and features like blink frequency and EAR are extracted. The LCD interface is configured to display a warning message such as "Drowsiness detected" when signs of fatigue are recognized. An alert system is also implemented to trigger sound or vibration alerts to wake the driver; if

the driver fails to respond, the system initiates vehicle control actions such as slowing down or stopping. Finally, the entire system is tested under both simulated and real-world driving conditions to assess its accuracy, response time, and overall reliability.

1.6 Scope and Limitations of the Study

The scope of this project is limited to the development of a prototype system for detecting driver drowsiness using facial feature analysis and vehicle-based sensors. The system is designed for use in personal and commercial vehicles and focuses on real-time detection and control. This project aims to develop a driver drowsiness detection system and a preprocessing application for the normalization of the video frame which will be run on a Raspberry pi 4. The driver drowsiness detection system would focus solely on facial recognition and heart rate sensing for real-time drowsiness detection using a model.

Some features will not be included within the scope of this project, and they include: The vehicle's infotainment system's proprietary protocols and physical sensors present in real-world vehicles. Also, some features of the ADAS such as Lane departure warning and Adaptive Cruise Control will not be included because these features can only be demonstrated in real-world vehicles. Also, the system's accuracy may be affected by lighting conditions, camera quality, and driver facial obstructions (e.g., sunglasses), the system is not designed to replace human judgment entirely but to assist drivers in recognizing and responding to drowsiness. Finally, the project does not address other causes of accidents, such as drunk driving or mechanical failures.

1.7 Project Report Organization

This project report is organized into five chapters:

Chapter One: Provides an introduction to the project, including the background, significance, problem statement, aim and objectives, methodology, scope, and limitations.

Chapter Two: Presents a literature review of existing drowsiness detection systems, including computer vision-based, physiological, and vehicle-based approaches.

Chapter Three: Describes the system design, including hardware and software components, data acquisition, and drowsiness detection algorithms.

Chapter Four: Discusses the implementation, testing, and evaluation of the system.

Chapter Five: Concludes the project, summarizing the findings, achievements, and recommendations for future work.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

Long highway driving increases physical fatigue and reduces a driver's attention span, leading to microsleep, delayed reaction, and impaired judgment—major contributors to road accidents. This chapter explores the key behavioral and physiological indicators of drowsiness, such as prolonged eye closure, yawning, and irregular steering. It then reviews three primary detection methods—vision-based, physiological, and vehicular—highlighting their strengths and limitations. A unified system that merges real-time detection with automated responses like alerts or vehicle control is proposed, supported by technologies including EEG wearables and deep-learning edge devices. The chapter concludes with recent research highlights and emerging innovations like sensor fusion and transformer models aimed at improving detection accuracy and driver safety.

2.2 Key Indicators of Driver Drowsiness

Both subjective self-report measures and observable physiological and behavioral indicators can be used to measure drivers' levels of drowsiness. While objective signs offer externally verifiable indicators that can be automated in detection systems, subjective measures offer a rapid, driver-reported assessment of sleepiness.

2.2.1 Subjective Sleepiness Scales

One of the most popular tools for self-rating sleepiness is the Karolinska Sleepiness Scale (KSS). Drivers are asked to rate their level of alertness on a nine-point scale, with 1 denoting "extremely alert" and 9 denoting "very sleepy, great effort to keep awake." KSS scores are an accurate indicator for determining system warning thresholds because of the numerous studies that show a strong correlation between them and objective performance degradations like increased reaction times and steering variability [9]. The KSS is supplemented by the Wierwille–Ellsworth Five-Level Scale, which divides drowsiness into five observable levels, from "not drowsy" to "extremely drowsy," based on overt behaviors such as frequent yawning, slow eyelid closures, and head nodding. By using

this behavioral grading, system designers can directly link intervention strategies to visible signs, making sure that speed-control commands and alerts are given at the right times in the drowsiness progression [9].

Table 1: Karolinska Sleepiness Scale

Scale	Verbal Description
1	Extremely alert
2	Very alert
3	Alert
4	Fairly alert
5	Neither alert nor sleepy
6	Some signs of sleepiness
7	Sleepy, but no effort to keep alert
8	Sleepy, some effort to keep alert
9	Very sleepy, great effort to keep alert

Table 2: Wierwille and Ellsworth Drowsiness Scale.

Levels	Verbal Description
1	Not drowsy
2	Slightly drowsy
3	Moderately drowsy
4	Significantly drowsy
5	Extremely drowsy

2.2.2 Objective Physiological and Behavioral Signs

In addition to self-reports, objective indicators of deteriorating alertness include increased blink duration, yawning frequency, head-pose drift, prolonged eye closures (often assessed using the Percentage of Eye Closure, or PERCLOS), and steering corrections [9]. Even earlier warnings are provided by physiological signals, such as changes in heart-rate variability or shifts in the EEG's theta and alpha power bands, but these usually call for wearable sensors. Vision-based systems concentrate on behavioral indicators, extracting

facial landmarks and computing metrics such as the Eye Aspect Ratio (EAR) using a camera mounted in the cab. When combined with facial analysis, vehicle signals, such as irregular steering inputs and lane-departure patterns, further support drowsiness. These objective indicators work together to create automated detection algorithms that can track driver conditions over time and initiate interventions when necessary [9].

2.3 Drowsiness Detection Modalities

Drowsiness detection systems can be broadly categorized based on the type of signals or cues they monitor. These include physiological signals (like brain waves and heart rate), vehicular metrics (such as steering or acceleration patterns), and visual indicators (such as eye closure and head position). Each modality offers unique strengths, ranging from high accuracy to non-intrusiveness, but also comes with limitations. Selecting the appropriate approach depends on the balance between comfort, reliability, and implementation complexity. The following subsections discuss these modalities in detail, beginning with physiological techniques.

2.3.1 Physiological Approaches

Physiological techniques provide some of the earliest and most accurate markers of drowsiness by directly accessing the body's signals. They usually need sensors to be in contact with the driver, though, which may affect comfort and wearability over time. Figure 2 illustrates a general framework for physiological parameters-based drowsiness detection systems. The process begins with the collection of datasets from image and video sources, which are then preprocessed through cleaning and converted into a database. Following this, features are extracted using various physiological sensors such as PPG, ECG, EEG, and somatic sensors. These extracted features undergo data transformation—primarily normalization—to ensure uniform scaling. The processed data is then evaluated using machine learning models like SVM, CNN, and HMM, with performance metrics such as F1 score, recall, and accuracy guiding the final decision. Based on the classification output, the system determines whether the driver is drowsy or alert.

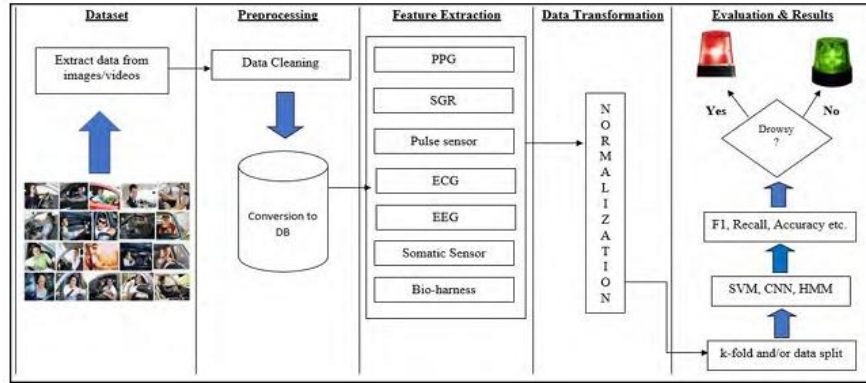


Figure 2: Framework of Physiological Parameters-Based Techniques

i. EEG-Based Systems

The gold standard for identifying shifts from alertness to drowsiness is still electroencephalography (EEG). Traditional multi-channel EEG systems track changes in certain frequency bands, especially the increase in theta power (4–7 Hz) and decrease in alpha power (8–13 Hz) that coincide with the onset of drowsiness. When spectral features are analyzed using wavelet decomposition or the Fast Fourier Transform, several laboratory studies report detection accuracies exceeding 90 percent [2]. However, these high accuracies require continuous impedance checking and appropriate electrode placement across the scalp, which are impractical for regular truck driving.

A lightweight, wearable EEG headset records brain signals, a sparse representation algorithm extracts the most discriminative power-spectral coefficients, and an ECU-embedded controller automatically applies the brakes or decelerates when prolonged drowsiness is detected [16]. Zhang et al. propose a three-stage system to bridge the gap between laboratory precision and real-world applicability. This architecture demonstrated that early detection can be directly linked to safe, automated interventions by successfully reducing vehicle speed within 1 second of confirmed drowsiness events in road-test validations. However, reducing the quantity and intrusiveness of electrodes is essential for driver acceptance.

Figure 3 illustrates the core components and workflow of an EEG-based drowsiness detection system. The process begins with the subject wearing an EEG cap embedded with multiple electrodes positioned across the scalp to capture electrical signals from the brain. These electrodes detect potential changes associated with neural activity, which are then

amplified to enhance signal clarity. The amplified signals are transferred to a processing unit, where various computational techniques are applied to extract meaningful patterns. These patterns—such as increased theta activity and reduced alpha waves—are analyzed to determine the driver's alertness level. This workflow is fundamental to many physiological-based detection systems due to EEG's sensitivity to changes in mental state.

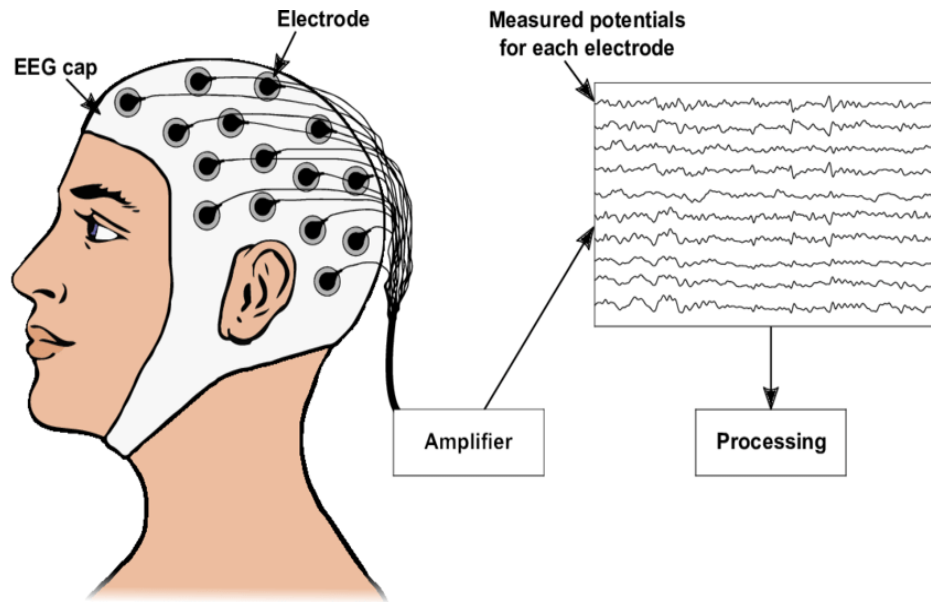


Figure 3: EEG components and workflow

ii. EOG and Heart Rate Variability (HRV)

The corneo-retinal potential, a readily detectable stand-in for eyelid movements, is measured by electrooculography (EOG) while drivers blink or close their eyes. As drowsiness increases, blink duration and frequency increase significantly. This is exploited by Pillay et al., who run an EOG blink-duration classifier solely on an ARM Cortex-M low-power microcontroller, enabling real-time inference without the need for large PCs or GPUs [20]. Their method uses less than 50 mW and achieves detection latencies of less than 100 ms, demonstrating that basic biosignal features can be used in deployments with limited resources.

Another non-invasive indicator of changes in the autonomic nervous system during sleepiness is Heart Rate Variability (HRV), which is obtained from photoplethysmography (PPG) sensors on the wrist or chest. In controlled tests, it has been demonstrated that HRV

metrics that are time-domain (like RMSSD) and frequency-domain (like LF/HF ratio) can differentiate between alert and drowsy states with an accuracy of roughly 95% [4]. However, driver movement or perspiration can compromise PPG signals, which are sensitive to motion artifacts and require strong filtering and sensor contact.

iii. Graph-Based EEG Inference

More effective EEG inference schemes have been sparked by recent developments in graph neural networks (GNNs). In two distinct graph streams, Huang et al. present a dual-graph-convolutional network (Dual-GCN) that encodes both spatial (electrode adjacency) and functional (signal-correlation) relationships. The model is compressed to less than 200 KB through channel- and neuron-level pruning, allowing for sub-second drowsiness inference on a Raspberry Pi 4 [21]. Compared to fully connected networks, this graph-based method reduces computational load and increases robustness to missing or noisy channels, which is a significant step toward embedded, non-intrusive EEG monitoring in car cabins.

2.3.2 Vehicular Approaches

Through the analysis of patterns in the vehicle's control and interaction with the road, vehicular approaches indirectly infer driver drowsiness. They provide a non-intrusive method of tracking driver vigilance because they only use onboard sensors and cameras, not wearable or body-attached technology. The two main subcategories of vehicular methods are the addition of fatigue awareness to Adaptive Cruise Control (ACC) and the analysis of steering and lane-keeping behavior.

i. Steering and Lane-Keeping Behavior

Fine motor control deteriorates when a driver's level of alertness starts to decline. One example is "micro-corrections" in steering, where the driver makes sudden, tiny reversals or overcompensates to maintain lane centering rather than making gradual, continuous adjustments. Thus, steering angle velocity and acceleration time-series analyses can be used as stand-ins for cognitive errors. When paired with statistical filtering techniques, Ramzan et al. find that the frequency and variance of these erratic corrections increase significantly in sleepy drivers, with detection accuracies of approximately 85–90% [4].

Concurrently, forward-facing cameras are used in computer-vision-based lane-departure monitoring to identify the visible lane markings on either side of the car. Inattention or microsleep is indicated by a sudden or gradual drift toward the lane boundary that is not corrected within a predetermined time window. False positives, however, can be caused by severe road curvature, faded paint, or bad weather. In order to counteract this, hybrid systems combine parallel eye-tracking and lane-departure alarms. If the car drifts but the driver's eyes stay firmly on the road, the alert may be suppressed; on the other hand, drift combined with closed or diverted eyes verifies a drowsiness event [4]. While maintaining sensitivity to actual fatigue, this combination of visual and vehicular cues drastically lowers the number of needless warnings.

ii. Adaptive Cruise Control (ACC) Augmentation

Driver state information can be added to adaptive cruise control, which was first created to maintain a safe following distance by automatically modifying the brakes and throttle in response to lead-vehicle behavior. Campos-Ferreira et al. incorporate real-time drowsiness scores into the ACC control loop, which are obtained from physiological or facial sensors. The ACC smoothly increases following distance and enforces a more conservative speed profile when the system determines that the driver's responsiveness (such as response to steering corrections or acceleration prompts) has dropped below a predetermined threshold [3].

This provides an extra margin of safety because, in reality, a tired driver does not have to manually disengage cruise; instead, the car automatically reduces its set speed or increases the distance to the car in front of it. The ACC can progressively slow down the car to a safe minimum speed or even bring it to a controlled stop if the driver doesn't re-engage, as shown by persistently high drowsiness metrics. This method combines comfort, convenience, and safety by utilizing the vehicle's current control infrastructure to provide fatigue mitigation without requiring extra driver input.

2.3.3 Behavioral (Vision-Based) Approaches

In-cab cameras are used in behavioral approaches to continuously monitor the driver's face and identify signs of drowsiness. These techniques, which strike a useful balance between intrusiveness and reliability, provide high accuracy without the need for direct

contact sensors by concentrating on facial landmarks and dynamic changes in the eye and mouth regions.

i. Eye Aspect Ratio (EAR) and PERCLOS

The Eye Aspect Ratio (EAR), which measures the ratio of vertical to horizontal eye landmark distances, is among the most straightforward but efficient metrics. Long-term eye closure, or PERCLOS (percentage of eye closure), is indicated by a persistent decline in EAR and is highly associated with microsleep events. Real-time EAR tracking systems have shown detection accuracies of over 98% in controlled experimental settings, with false-alarm rates of less than 2% when appropriate landmark tracking and illumination compensation are used [4]. EAR calculation's low computational cost makes it ideal for embedded platforms, enabling ongoing monitoring without taxing CPU power. Figure 4 presents a typical architecture for drowsiness detection systems based on eye state analysis. The process begins with input from a video stream, live camera, or existing database, followed by the detection of the driver's face within each frame. Once the face is localized, the eye region is extracted as the primary area of interest for drowsiness assessment. Features from the eye region are then extracted using a Convolutional Neural Network (CNN), which captures subtle patterns such as eye closure and blink frequency. These features are passed into a classifier—such as a Softmax layer or a Radial Basis Function Neural Network (RBFNN)—to determine whether the driver is drowsy or alert. This flow highlights the effectiveness of vision-based approaches in evaluating driver fatigue in real time.

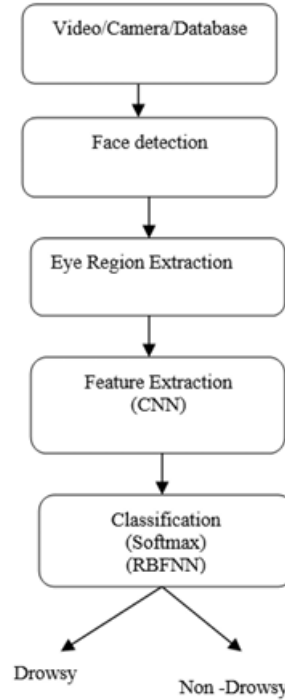


Figure 4: Architecture of Drowsiness Detection based on Eye State

ii. Deep Learning Pipelines

Many systems use convolutional neural networks (CNNs) on top of traditional face-detection algorithms to improve robustness in difficult situations. To classify each eye as "open" or "closed," an early method uses a shallow CNN in conjunction with Viola-Jones for fast face/eye region proposals [11]. Since the initial Haar cascades have trouble localizing facial features, this pipeline performs worse in low light or high glare conditions, even though it can achieve real-time performance on desktop GPUs.

By moving inference to specialized edge hardware, recent developments overcome these constraints. On an NVIDIA Jetson Nano, Florez et al. deploy a CNN that combines EAR and Mouth Aspect Ratio (MAR) metrics, allowing for the simultaneous detection of yawns (via MAR) and microsleeps (via EAR) [17]. Their system maintains over 96% accuracy even when illumination changes thanks to training on a variety of datasets, including those with glare and nighttime conditions.

Rapid head movements and occlusion resistance have been further enhanced by transformer-based architectures. By applying self-attention mechanisms to image patches surrounding the eyes, Hassan et al. allow the model to dynamically "focus" on the most

informative areas, even when the face is partially obscured by hands, glasses, or partial shadows [18]. With sub-150 ms inference times when deployed on an Edge TPU, this transformer is suitable for real-time automotive applications.

Six lightweight machine learning classifiers—Logistic Regression, Naive Bayes, K-Nearest Neighbors, Decision Tree, Random Forest, and Multilayer Perceptron (MLP)—are assessed on both Jetson TX2 and Nano boards in a related line of work by Lee et al. [19]. According to their benchmarking, MLP outperforms simpler models in situations with rapid head turns or partial occlusions, providing the best trade-off between accuracy, F1-score, and inference latency under 200 ms. Figure 5 illustrates the architecture of a Convolutional Neural Network (CNN), divided into Feature Extraction and Classification phases. The Feature Extraction phase, comprising Convolution and Pooling layers, automatically extracts hierarchical features from the input, like edges and textures. The Classification phase then uses Fully Connected layers to interpret these extracted features and produce a final Output classification, representing the network's prediction. This structured approach allows CNNs to excel in tasks like image recognition by learning patterns from raw data.

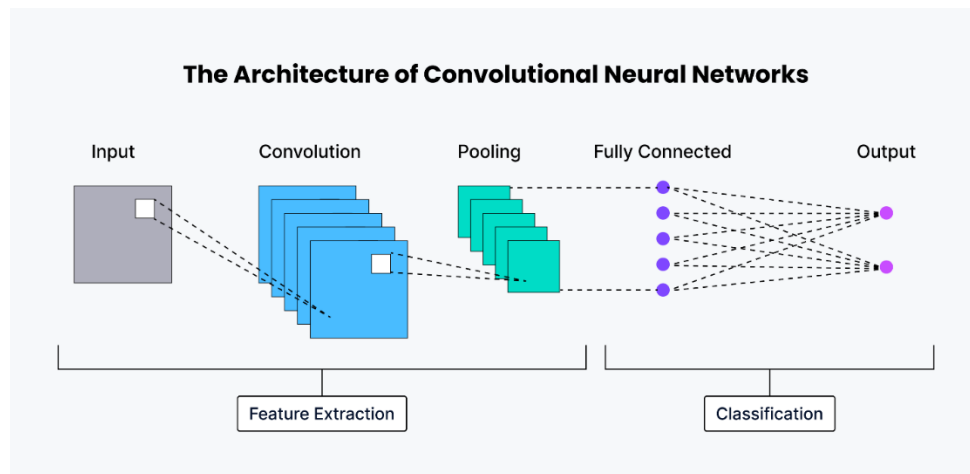


Figure 5: CNN Architecture

iii. Fuzzy Inference Engines

Fuzzy-logic engines offer an interpretable decision layer to manage uncertainty and integrate multiple behavioral cues. A Mamdani fuzzy inference system that combines

CNN-based eye-state metrics with EOG-derived blink duration is proposed by Kumar et al. [22]. Their system gracefully deteriorates when one sensor stream fails by defining linguistic variables (such as "ShortBlink," "MediumBlink," "LongBlink," and "ClosedEyes") and creating fuzzy rules that translate combinations of blink and eye-state inputs to output degrees of drowsiness. This hybrid EOG-vision method maintains over 95% detection accuracy with a false-positive rate of less than 5% in tests conducted in various lighting conditions, with and without facial hair [22]. Figure 6 presents the structure of a fuzzy logic system, highlighting its key components and their interactions. The system begins with a crisp input, which is converted into a fuzzy value by the fuzzification inference unit. This unit is connected to the knowledge base, which provides essential rules for decision-making. The decision-making unit processes the fuzzy values using stored rules and sends them to the defuzzification inference unit, which converts them back into a crisp output. The final crisp output represents the system's decision, demonstrating how fuzzy logic handles uncertainty in real-world applications.

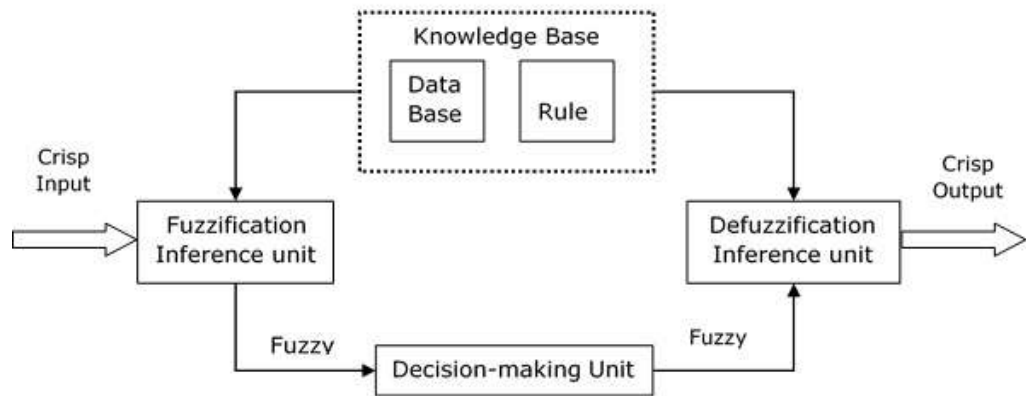


Figure 6: Fuzzy Inference System

2.4 Integrated Warning and Speed-Control Frameworks

To prevent accidents, a strong system must convert the detection of driver drowsiness into prompt, phased interventions. To make sure the driver and surrounding infrastructure react appropriately, integrated frameworks usually combine networked notifications, automated vehicle control measures, and staged alerts.

2.4.1 Two-Stage Intervention

A widely adopted strategy is the two-stage intervention model, which escalates responses according to driver reaction:

I. Stage 1 – Immediate Alerting:

The system emits a clear warning when a predetermined drowsiness threshold—which can be determined by physiological, behavioral, or vehicle metrics—is exceeded. This could look like:

- i. **Auditory signals**, such as a loud beep or a spoken instruction, like "Please take a break," that cut through background cabin noise.
- ii. **Haptic feedback**, transmitted through vibrations in the seat or steering wheel, which can attract the driver's attention even when the driver is distracted or their auditory channels are muted.

By providing a sudden, unexpected stimulus, early alerts use the ritualized awakening principle to temporarily raise the driver's arousal level and possibly prevent microsleep from starting.

II. Stage 2 – Automated Deceleration:

The system switches to active vehicle control if there is no driver response (such as a steering correction, manual pedal input, or button press) within a configurable timeout, which is typically set between three and ten seconds. The truck can be safely slowed down or brought to a complete stop if needed by coordinating throttle reduction and light braking to reduce vehicle speed at a controlled rate (e.g., 0.5–1.0 m/s² deceleration).

By combining ECU braking commands with a wearable EEG headset for early detection, Zhang et al. validate this method, demonstrating that it prevents lane departures during road tests and achieving consistent speed reductions within one second of verified drowsiness events [16]. In a similar vein, Sengar et al.'s VigilEye platform reduces the likelihood of accidents in simulation scenarios by combining PERCLOS monitoring with CAN-bus injection of brake-control messages to enforce deceleration when eye-closure metrics surpass thresholds [15].

2.4.2 Vehicle-to-Everything (V2X) Alerts

Modern fleets gain from using V2X communication protocols to broadcast drowsiness events to external entities, in addition to in-cab reactions. A truck's onboard system can send a standardized drowsiness message (such as an ETSI CAM with an extended data payload) to the following when it detects prolonged fatigue, which is frequently verified in Stage 2:

- i. **Nearby infrastructure**, such as roadside units (RSUs), which allow dynamic warning signs or adaptive traffic-signal timing to notify oncoming vehicles.
- ii. **Fleet management centers**, which can plan required breaks, send support vehicles, or reroute the driver to the closest rest area.

V2X integration enables coordinated responses, as shown by Adnan Yusuf et al. For instance, a roadside RSU that receives a drowsiness alert can turn on flashing beacons on the shoulder to safely direct the exhausted driver off the highway [8]. The system's protective reach is extended beyond the individual vehicle through such networked interventions.

2.4.3 Adaptive Cruise Control (ACC) Enhancement

In order to maintain safe following distances, adaptive cruise control systems automatically modify the throttle and brakes. Without direct intervention, the car can proactively adjust to waning driver attentiveness by integrating drowsiness tests into the ACC control loop:

- i. **Dynamic Speed Profiling:** In order to reduce collision risk in the event that the driver does not react to lead-vehicle behavior, the ACC gradually lowers its target speed (for example, by 5–10 km/h) and increases headway distance (for example, from 2 seconds to 3-second time gap) when drowsiness confidence metrics exceed a warning threshold [3].
- ii. **Automated Safe-Stop Mode:** In the event that the driver's condition worsens further—as indicated by consistently high drowsiness scores or the lack of manual overrides—the ACC switches to Safe-Stop mode, which stops the truck in its current lane or, if it has one, directs it to the shoulder of the slow lane.

In order to reduce the risk of rear-end collisions during lengthy highway segments, Campos-Ferreira et al. integrated facial drowsiness outputs from on-board cameras into the ACC module. They found that fatigued drivers automatically maintained longer following distances and experienced fewer abrupt braking events [3]. By subtly altering driving parameters in response to fatigue, this smooth augmentation maintains the driver's comfort and lessens cognitive load.

2.5 Comparative Review of Related Works

In order to create comprehensive Driver Drowsiness Detection and Speed Control Systems, recent research emphasizes the integration of physiological, behavioral, and vehicular data—often via AI models. These studies show how to combine automated interventions like ACC adjustment and ECU-driven deceleration with real-time inference on edge platforms (like the Jetson Nano and Raspberry Pi). Key implementations and their performance metrics are compiled in Table 2.3.

Table 3: Review of Related Works

Ref.	Paper	Technique	Process	Result	Gap
[9]	A Review of Recent Developments in Driver Drowsiness Detection Systems	Thermal imaging + SVM/KNN	Infrared camera	90 % acc., 92 % sens.	Variable with environment
[11]	Deep CNN: A machine learning approach for driver drowsiness detection based on eye state	Viola-Jones + 4-layer CNN	Desktop GPU	Real-time; ~95 % acc.	Sensitive to lighting
[16]	A Vehicle Active Safety Model: Vehicle Speed Control Based on Driver Vigilance Detection Using Wearable EEG and Sparse Representation	Wearable EEG + sparse spectral features + ECU control	Vehicle ECU + wearable EEG	Validated in simulation & road tests	Intrusive wearable sensors
[17]	A Real-Time	CNN fusion	NVIDIA	>98 % acc.	Edge

	Embedded System for Driver Drowsiness Detection Based on Visual Analysis of the Eyes and Mouth Using Convolutional Neural Network and Mouth Aspect Ratio	of EAR & Mouth Aspect Ratio	Jetson Nano		compute requirements
[18]	Real-time driver drowsiness detection using transformer architectures: a novel deep learning approach	Transformer for eye-state classification	Edge TPU / Jetson	Real-time edge inference	Model complexity
[19]	Real-Time Driver Drowsiness Detection and Classification on Jetson Nano and Jetson TX2 Embedded Systems Using Machine Learning Algorithms	Logistic, NB, KNN, DT, RF, MLP comparative study	Jetson TX2 / Nano	MLP: 91 % acc., 91 % F1	Hardware-specific tuning
[20]	Real-Time Detection of Drowsiness Among Vehicle Drivers: A Machine Learning Algorithm for Embedded Systems	EOG blink-duration classifier on MCU	ARM Cortex-M	Feasible MCU inference	Limited to blink features
[21]	LDGCN: An Edge-End Lightweight Dual GCN Based on Single-Channel EEG for Driver Drowsiness Monitoring	Pruned dual-GCN on single-channel EEG	Raspberry Pi 4	Sub-second inference	Requires EEG hardware
[22]	Driver Drowsiness Multi-Method Detection for Vehicles with Fuzzy Decision Algorithm	Fuzzy logic fusion of EOG & image features	Embedded ARM Platform	>95 % acc.	Complexity of fuzzy rule design

[24]	Challenges of Driver Drowsiness Prediction: The Remaining Steps to Implementation	Survey of deployment challenges	—	—	Highlights need for naturalistic data
[25]	A Review of Driver Drowsiness Detection Systems: Techniques, Advantages and Limitations	Comparative review of physiological, image, vehicular	—	—	Calls for balanced intrusiveness

2.6 Challenges and Emerging Directions

Despite significant advancements in Driver Drowsiness Detection Systems (DDDS), several critical challenges persist that hinder their widespread adoption and real-world efficacy. These include the trade-off between sensor intrusiveness and driver comfort, often associated with physiological monitoring. Additionally, environmental factors like variable lighting and diverse driving conditions significantly impact the reliability of vision-based systems. Addressing these limitations is crucial for developing.

2.6.1 Sensor Intrusiveness vs. Driver Comfort

Some of the earliest and most accurate drowsiness indicators are provided by physiological monitoring methods like EEG and EOG, but they frequently call for direct contact sensors on the driver's scalp or around the eyes. Although theta-band surges, which are a sign of microsleeps, can be detected by wearable EEG headsets in a matter of a few seconds, their requirement for multiple electrodes and gel-based contacts may cause discomfort, skin irritation, and resistance to prolonged use [16]. Similar to this, EOG sensors placed close to the eyes offer precise measurements of blink duration and eye closure; however, their close proximity to the driver's face may be viewed as obtrusive and may impede normal head movement [20]. According to recent surveys, driver acceptance will continue to be a major obstacle to widespread adoption unless sensor form factors significantly reduce in size, become lighter, and become more seamlessly integrated into commonplace accessories (like smart glasses or caps) [23].

2.6.2 Lighting Conditions and Environmental Variability

Variable lighting, shadows, and occlusions are challenges for vision-based behavioral systems, which use cameras to track facial landmarks and calculate metrics like EAR or MAR. Conventional CNN and cascade-based detectors are challenged by day-night transitions, tunnel passages, backlighting from oncoming traffic, and dynamic weather conditions [11]. With their self-attention mechanisms, transformer-based architectures have proven to be more resilient, concentrating computational resources on the most informative image patches even when partially obscured by hand movements or sunglasses [18]. Although they add more hardware complexity and expense, thermal fusion techniques that blend visible-light feeds with infrared imaging provide an additional route to reliable performance in low-light conditions. Research on optimizing these sophisticated models for real-time edge deployment is still ongoing as they develop [17], [23].

2.6.3 Dataset Standardization and Deployment Gaps

Most drowsiness detection studies test their techniques on small, controlled datasets that were recorded in instrumented cars or simulators. The entire spectrum of driver demographics, cabin lighting variations, road geometries, and cultural driving behaviors that are encountered "in the wild" are not covered by this constrained scope. Models run the risk of overfitting to laboratory settings and performing poorly in real-world deployments in the absence of extensive, naturalistic driving datasets that are annotated with synchronized physiological, behavioral, and vehicular signals. On-road trials are still rare because of logistical, legal, and privacy issues, and efforts to gather and make publicly available such multimodal corpora are still in their infancy [24]. Improving the generalizability and chances of regulatory approval of fatigue-monitoring systems will require filling in these data gaps.

2.6.4 Hybrid Sensor-Fusion Architectures

Hybrid systems combine behavioral, physiological, and vehicular cues within adaptive decision engines to get around the drawbacks of any one modality. For instance, fuzzy-logic frameworks create graded drowsiness scores that gradually deteriorate when one sensor stream fails by combining EOG blink durations with image-based eye-state

probabilities [22]. For more reliable inference in noisy environments, graph-based neural networks can also incorporate steering-behavior features with EEG channel correlations. By combining the complementary advantages of each data source—contextual nuance from vision, consistency from vehicular telemetry, and early warning from physiology—these architectures provide real-time performance that satisfies usability and safety standards [25]. With further advancements in dynamic sensor weighting, edge compute optimization, and lightweight model pruning, hybrid approaches could become the mainstay of next-generation driver fatigue detection.

2.7 Chapter Summary

The range of drowsiness indicators has been covered in this chapter, along with physiological, vehicle, and behavioral detection techniques and comprehensive frameworks that combine automated speed control and real-time alerts. Trade-offs between accuracy, intrusiveness, and computational demands are highlighted in a comparative study of state-of-the-art implementations. Ultimately, the following major issues are noted as important directions for further study and real-world application in commercial trucking: sensor comfort, environmental variability, data gaps, and hybrid system architectures.

CHAPTER 3

SYSTEM ANALYSIS AND DESIGN

3.1 Introduction

This chapter outlines the design and implementation of a Driver Drowsiness Detection and Speed Control system using a Raspberry Pi 4 prototype. It discusses key functional requirements such as continuous alertness monitoring, a tiered warning system, and automated speed reduction. The hardware setup—including the camera, alert modules, sensors, and speed simulator—is described alongside high-level diagrams to show system integration. The software architecture covers real-time data acquisition, preprocessing, inference, and control logic. Finally, design decisions are justified in terms of reliability, safety, and scalability for real-world vehicle applications.

3.2 System Requirements

This section describes the function and essential features of each piece of hardware used in the Driver Drowsiness Detection and Speed Control prototype based on the Raspberry Pi.

3.2.1 Raspberry Pi 4

The system's central processing unit is the Raspberry Pi 4. Real-time video capture, facial landmark extraction, and neural network inference are made possible by its quad-core ARM Cortex-A72 processor and up to 4 GB of RAM. Its GPIO header offers digital I/O for alerting and motor-control signals, and its numerous USB ports and Gigabit Ethernet interface facilitate camera connectivity and remote updates. Figure 7 shows the Raspberry Pi.

3.2.2 Raspberry Pi USB Camera

The Raspberry Pi USB Camera serves as the primary imaging device in the system. Connected via one of the Pi's USB 2.0 or 3.0 ports, it enables high-resolution video capture essential for real-time image processing tasks. Unlike native CSI-based Pi cameras, the USB interface allows compatibility with a wide range of third-party webcams and industrial cameras, providing flexibility in resolution, frame rate, and lens configuration. The camera streams video directly to the Pi's processor, where frames are

used for tasks such as object detection, facial recognition, or environmental monitoring. Its plug-and-play support under the Video4Linux (V4L2) driver framework ensures quick integration and reliable performance within Linux-based application. Figure 7 shows the Raspberry Camera.

3.2.3 12 V Battery Input Connector

The prototype receives unregulated DC power from this terminal block, which takes the vehicle's 12 V battery supply. It supplies the buck converter's raw input as well as a handy power source for high-current accessories like motor simulators and buzzers. Figure 7 shows the Battery connector.

3.2.4 USB-A Port

Firmware and software updates, debugging, and thumb drive data export are all possible with a standard USB-A connector. When extra camera modules are needed for vision processing, it also supports USB webcams.

3.2.5 Buck Converter (LM2596S-5V)

For the Raspberry Pi and related sensors, the LM2596S-5V switch-mode regulator reduces the 12 V battery input to a steady 5 V rail. Even during peak load transients, it keeps output ripple below 50 mV by using a 47 μ H inductor (L3) and bulk/smoothing capacitors. Figure 7 shows the Buck converter.

3.2.6 Inductor (47 μ H)

The energy-storage component of the buck converter is made up of the 47 μ H inductor and the LM2596S regulator. In order to maintain low power dissipation in a mobile environment, it smoothes current pulses and permits high-efficiency voltage conversion.

3.2.7 Bulk Capacitors (4700 μ F/16 V)

Bulk energy storage is provided by these electrolytic capacitors, which are located at the converter output. They ensure the 5 V rail stays rock-solid under dynamic conditions by filtering low-frequency voltage fluctuations and supplying large current bursts during processor-intensive tasks.

These ceramic capacitors stop high-frequency noise on the 5 V supply when they are placed near the Raspberry Pi's power pins and other integrated circuits. They stop transients from digital switching from coupling into analog circuits that are sensitive.

3.2.9 Smoothing Capacitor (220 μ F/35 V)

This electrolytic capacitor is mounted on the buck converter's 12 V input side. It protects the regulator from abrupt changes in supply by absorbing input voltage dips and filtering out noise from the vehicle battery line.

3.2.10 Buzzer

Upon detecting drowsiness, the stage-1 auditory alert is delivered by the electromechanical buzzer. It's powered by a GPIO-controlled transistor and produces a loud tone that cuts through cabin noise, causing the driver to refocus before the vehicle automatically decelerates. Figure 7 shows the Buzzer.

3.2.11 Indicator LEDs

Complementary stage-1 alerts are provided by a collection of visual indicators. To ensure that the warning is visible even when the buzzer is muted, LEDs mounted on the dashboard light up in different patterns or colors to indicate increasing levels of drowsiness.

3.2.12 Blocking Diode (1N4007)

The power input is shielded from reverse polarity by the 1N4007 general-purpose rectifier. It makes sure that unintentional battery connections don't harm the Raspberry Pi or the buck converter.

3.2.13 Schottky Diode (SS34)

This SS34 Schottky diode, which is positioned in the switching path of the buck converter, improves conversion efficiency and lowers heat generation by offering quick recovery and low forward voltage drop. Figure 7 shows the Schotky diode.

3.2.14 Transistors (2SC1815)

The buzzer and warning LEDs are driven by these NPN bipolar junction transistors, which also act as level shifters. With the help of current-limiting resistors to ensure safe

operation, they convert the Pi's 3.3 V GPIO signals into 5 V switching commands. Figure 7 shows the Transistors.

3.2.15 MOSFET Driver (IRF4905)

The motorized speed simulator is controlled by the IRF4905 P-channel MOSFET, which receives gate-drive signals from the Raspberry Pi via a level-shifting transistor. Its high current rating and low $R_{DS(on)}$ allow for accurate throttle emulation without causing undue heat loss. Figure 7 shows the Mosfet.

3.2.16 Resistor Networks

This design's resistor network guarantees appropriate signal levels and device security. As base-current limiters for transistors Q1 and Q2, resistors R9 and R11 (each 1 k Ω) stop the Raspberry Pi's GPIO lines from drawing too much drive current. If the GPIO pin floats or the Pi loses power, a 10 k Ω pull-down resistor (R10) on the MOSFET gate ensures that the motorized speed simulator stays off. Lastly, the three 220 Ω resistors (R12, R13, and R14) control the current flowing to the indicator LEDs, ensuring that there is enough brightness for visibility while limiting power consumption and heat production. Figure 7 shows the Resistors.

3.2.17 I2C Led 1602 with Module.

The I2C 1602 LCD is a 16x2 character display module that uses the I2C communication protocol, reducing the number of GPIO pins required from 12 to just 2 (SDA and SCL). It features an onboard I2C controller (usually a PCF8574 chip) that interfaces with the main LCD controller (HD44780). The module operates on 5V and is compatible with devices like Arduino and Raspberry Pi. It's ideal for projects with limited GPIO availability. A potentiometer on the module allows adjustment of the display contrast. Figure 7 shows the LCD.

3.2.18 GY Max30102

The GY-MAX30102 is a compact biomedical sensor module used to measure heart rate and blood oxygen saturation (SpO₂). It integrates the MAX30102 chip, which combines two LEDs (red and infrared), a photodetector, optical elements, and low-noise electronics. The module communicates via the I2C protocol, making it easy to interface with microcontrollers like Arduino and Raspberry Pi. It is designed for wearable and portable

health monitoring applications. Proper finger placement and ambient light shielding are important for accurate readings. Figure 7 shows the GY Max30102.

3.2.19: Electric TT Magnetic DC Motor Gearbox dual shaft geared Axis

The Electric TT Magnetic DC Motor with Gearbox and Dual Shaft is a compact, low-voltage motor commonly used in DIY electronics, robotics, and small vehicle projects. It features a plastic gearbox that reduces speed while increasing torque, making it suitable for driving wheels or mechanical arms. The dual shaft design allows attachments on both sides—such as encoders or additional wheels—for balanced movement or feedback control. It operates typically at 3–6V DC and can be easily driven using an H-bridge motor driver. Its small size and affordability make it ideal for educational and prototype applications. Figure 7 shows the motor.

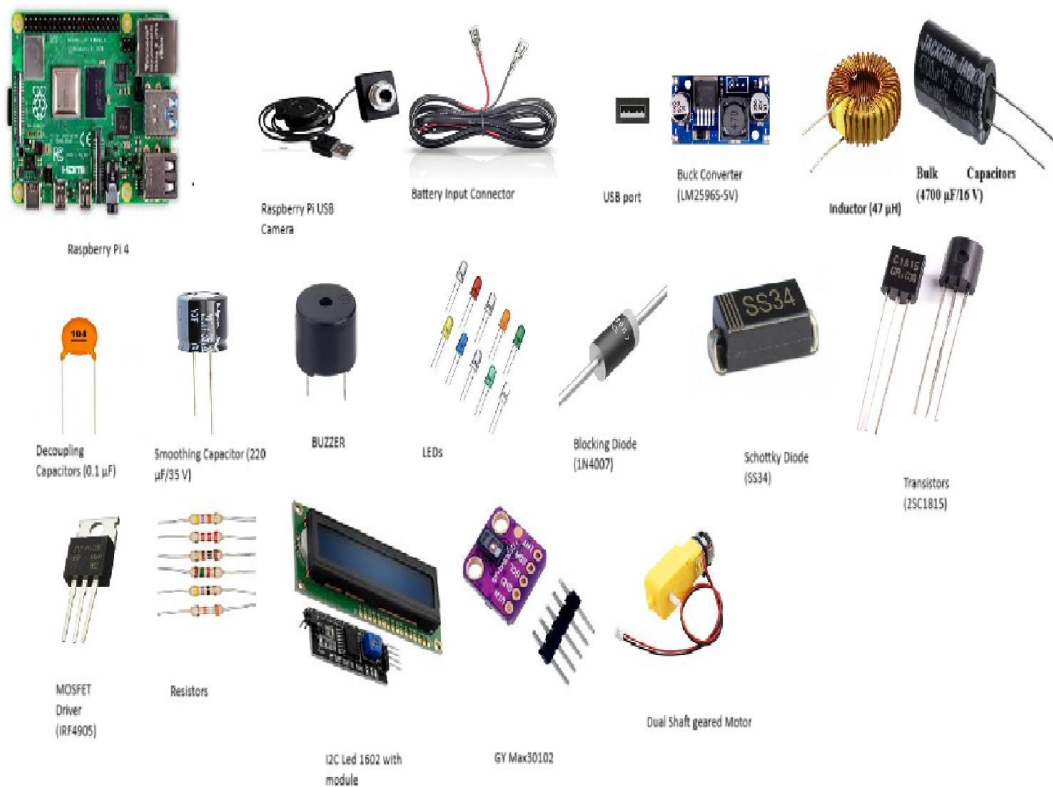


Figure 7: Pictures of Components

3.3 System Design Specification

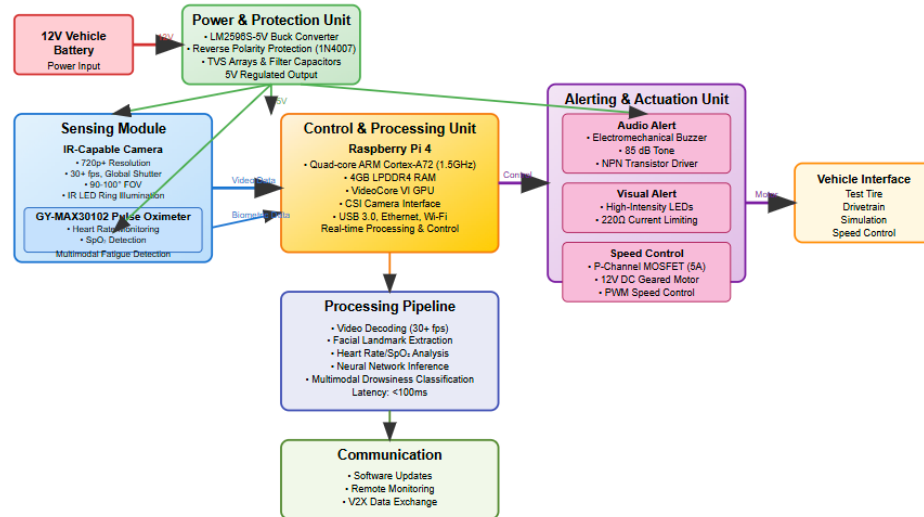


Figure 8: Design Specification

In order to implement the Driver Drowsiness Detection and Speed Control prototype, the system design specification offers detailed instructions for configuring both hardware and software components. It converts the functional requirements—automatic speed modulation, tiered alerting, and real-time fatigue monitoring—into precise design specifications that guarantee dependable performance in a range of driving circumstances. Figure 8 shows the design specification of the system including both the hardware and software.

There are two primary sections to the specification:

3.3.1 Hardware Design

Each subsystem's electrical, physical, and connectivity requirements are specified in the hardware design. Every component was chosen for its low latency operation, suitability for automotive settings, and compatibility with the Raspberry Pi 4 backbone. Important subsystems of hardware include:

i. Sensing Module

The sensing module, which is in charge of gathering all the raw data needed to deduce driver state, makes up the crucial front end of the drowsiness detection system. Its main component is an IR-capable camera that is mounted eye-level on the dashboard or steering column. This camera can be a USB webcam with infrared LEDs or a dedicated CSI-connected board camera. This arrangement minimizes parallax errors during head rotations and body sway while guaranteeing an unhindered view of the driver's face. In order to record quick eyelid movements and subtle facial expressions without motion blur, the camera must be able to capture at least 30 frames per second. It must also use a global shutter to remove rolling artifacts while the vehicle is moving. Both eyes can be reliably detected even if the driver looks sideways thanks to high resolution (at least 720p) and a broad field of view (roughly 90–100°). While an onboard IR LED ring offers uniform illumination without diverting the driver's attention, infrared sensitivity guarantees consistent performance in low light or at night.

ii. Control and Processing Unit

The Raspberry Pi 4 is at the center of the system because of its rich I/O capabilities, energy efficiency, and balance of processing power. With a maximum speed of 1.5 GHz, its quad-core ARM Cortex-A72 processor provides the throughput required for neural-network inference pipelines, facial landmark extraction, and real-time video decoding. Large intermediate data buffers and multi-threaded workloads are supported by the Pi's 4 GB of LPDDR4 RAM, which is essential for sustaining consistent frame rates of 30 fps or more. While the CSI camera interface and USB 3.0 ports offer low-latency connectivity to high-resolution infrared cameras, the onboard VideoCore VI GPU speeds up standard computer-vision tasks. Software updates, remote monitoring, and optional Vehicle-to-Everything (V2X) data exchange are made possible by gigabit Ethernet and dual-band Wi-Fi.

An LM2596S-5V buck converter, which steps down the vehicle's 12 V battery to a clean 5 V rail for the Raspberry Pi and all connected peripherals, maintains power stability. Thermal dissipation is reduced by the regulator's high switching efficiency (up to 92%), which keeps the average system power draw under 3 W even during bursts of peak

processing. Voltage transients are smoothed by a network of bulk ($4 \times 4,700 \mu\text{F}$) and decoupling ($0.1 \mu\text{F}$) capacitors, and reverse polarity and inductive kickback are prevented by Schottky and fast-recovery diodes. By ensuring that the Pi and its sensors function within strict voltage tolerances, this power-delivery subsystem allows for consistent inference latency below 100 ms and continuous fatigue detection over extended operation.

iii. Alerting and Actuation Unit

By providing unambiguous warnings and, if required, mimicking automated vehicle control, the alerting and actuation unit fills the gap between fatigue detection and driver intervention. When drowsiness thresholds are crossed, an electromechanical buzzer at the first level of response sounds an 85 dB tone, loud enough to cut through normal cabin noise. In order to isolate the Raspberry Pi from the buzzer's 12 V supply and guarantee dependable activation even in the presence of voltage fluctuations, the buzzer is powered by a Raspberry Pi GPIO pin that switches a level-shifting NPN transistor. A bank of high-intensity LEDs mounted on the dashboard provide visual alerts in addition to the auditory cue; each LED is connected to a 220Ω resistor to control current and balance brightness without producing too much heat. When drowsiness persists, the LEDs can quickly escalate warning patterns (such as flashing frequency) because they react to Pi signals in less than 10 ms.

The actuation element of the system kicks in if the driver doesn't react within the specified timeout. A P-channel MOSFET rated for at least 5 A continuous current controls a 12 V DC geared motor that is fitted with a test tire to simulate a vehicle's drivetrain. In order to convert the Pi's 3.3 V logic into the gate voltages needed for dependable high-side switching, the MOSFET gate is driven via a level-shifting transistor network. Pulse-width modulation (PWM) signals from the Pi are used to modulate the motor's speed during actuation, allowing for smooth throttle reduction profiles that simulate gradual deceleration. The MOSFET's snubber networks and protective flyback diodes across the motor terminals guard against voltage spikes harming the driver circuitry. When combined, these systems guarantee that warnings are clear, actuation is secure and regulated, and the changeover from warning to intervention is seamless and reproducible in test situations.

iv. Power and Protection Unit

Under the severe electrical conditions common to heavy vehicles, the power and protection unit makes sure that the delicate electronics in the drowsiness-detection system continue to function and not be harmed. A reverse-polarity blocking diode (1N4007) is placed right after the 12 V battery input connector to guard against damage in the event that the battery leads are inadvertently switched. A bank of decoupling ceramics ($2 \times 0.1 \mu\text{F}$) and bulk electrolytic capacitors ($4 \times 4,700 \mu\text{F}/16 \text{ V}$) downstream filter out high-frequency noise produced by other vehicle subsystems and absorb low-frequency voltage fluctuations. High-energy transient-voltage-suppression (TVS) arrays and fast-recovery diodes (SS34) clamp spikes to safe levels to prevent load-dump events, which are sudden surges above 40 V when the alternator load changes suddenly. Inference latency and sensor accuracy remain constant under all operating conditions thanks to the regulated 5 V rail that supplies the Raspberry Pi and sensors, which is kept within $\pm 5\%$ of nominal even during engine start/stop cycles or abrupt throttle changes.

3.3.2 Software Design

The modular architecture and interfaces needed to achieve performance and reliability goals are specified in the software design:

i. Data Acquisition Module

The Data Acquisition Module continuously records raw inputs for processing later on, acting as the system's front-line interface to all sensors. To make sure no crucial blink or microsleep event is missed, it launches specialized, real-time threads to poll the IR-capable camera at a minimum rate of 30 samples per second. It minimizes CPU overhead and jitter for video streams by using hardware-accelerated drivers, like V4L2 on the Raspberry Pi's CSI or USB interfaces, to transfer image capture and buffer management to the GPU and DMA engines. To maintain strict temporal alignment with packets, which arrive via I²C or SPI at matched sample rates, each frame is time-stamped upon arrival and enqueued into a lock-free ring buffer. In order to avoid overflow or underflow during periods of high load, the module dynamically modifies thread priorities while monitoring

buffer occupancy in real time. In order to ensure a stable and continuous stream of synchronized data for precise drowsiness inference, built-in health checks confirm sensor connectivity and data integrity. They do this by sending fault flags if frames are dropped or if electrode impedance thresholds drop below acceptable levels.

ii. Preprocessing Module

The Preprocessing Module prepares raw input data for real-time inference by cleaning, normalizing, and structuring it into formats compatible with the detection algorithms. For camera input, it applies grayscale conversion, histogram equalization, and spatial cropping to isolate the eye region while reducing noise and lighting variations. Sensor signals are filtered using digital smoothing and baseline correction to remove artifacts from motion or electrical interference. All data streams are then resized and normalized to meet model input constraints—such as fixed image dimensions or standardized physiological ranges—ensuring consistent and reliable behavior under varying conditions. This module guarantees that the subsequent inference engine receives high-quality, low-noise data with minimal latency.

iii. Control Logic Module

The Control Logic Module interprets inference outputs and converts them into system responses such as alerts or speed modulation. When a drowsiness state is detected—based on eye closure, blink rate, or signal anomalies—the module first activates warning mechanisms, including visual LEDs and an 85 dB buzzer, in accordance with configurable severity levels. If the driver remains unresponsive beyond a threshold time, it escalates to automated control by sending pulse-width modulation (PWM) signals to the motor to simulate gradual deceleration. Fail-safes are embedded to prevent accidental triggering, and a watchdog timer resets the system if outputs become erratic. The logic ensures timely and proportionate reactions to fatigue without overwhelming the driver or compromising system stability.

3.4 System Implementation

The process of turning the Driver Drowsiness Detection and Speed Control prototype from design into a functional system is broken down in this section. It addresses end-to-end testing, software deployment, sensor integration, and hardware assembly.

3.4.1 Hardware Assembly

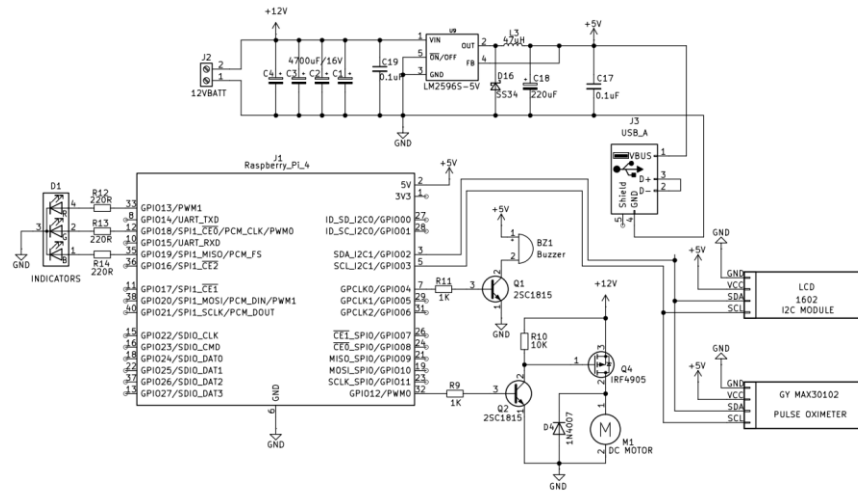


Figure 9: Schematic Diagram

The schematic diagram as seen in figure 9 illustrates a Raspberry Pi-based drowsiness detection and alert system that integrates multiple sensors and actuators to monitor a user's alertness. A 12V battery powers the system, and an LM2596S buck converter steps the voltage down to 5V, supplying the Raspberry Pi and other 5V components. The Raspberry Pi serves as the main controller, interfacing with peripherals via GPIO, I2C, and UART connections. A GY-MAX30102 pulse oximeter sensor is connected to the Pi via I2C to monitor heart rate and SpO₂ levels, which are critical for detecting signs of fatigue. A 1602 I2C LCD module is also connected to display real-time physiological data. The circuit includes three indicator LEDs driven by GPIOs through 220Ω resistors to provide visual feedback. A buzzer is connected through a 2SC1815 transistor (Q1) and is activated by a GPIO pin to alert the user when drowsiness is detected. A DC motor is controlled by an IRF4905 P-channel MOSFET (Q4), driven through a two-stage BJT switching configuration using Q2 and Q3 (both 2SC1815), which allows the Pi to handle

higher current switching indirectly. The system also includes decoupling capacitors (e.g., C14–C18) to smooth voltage lines and protect sensitive components. A USB-A port indicates a camera interface, likely used for facial or eye closure detection. The setup enables a multi-modal alert system, combining both visual and physiological inputs to trigger warnings or responses like motor shutdown.

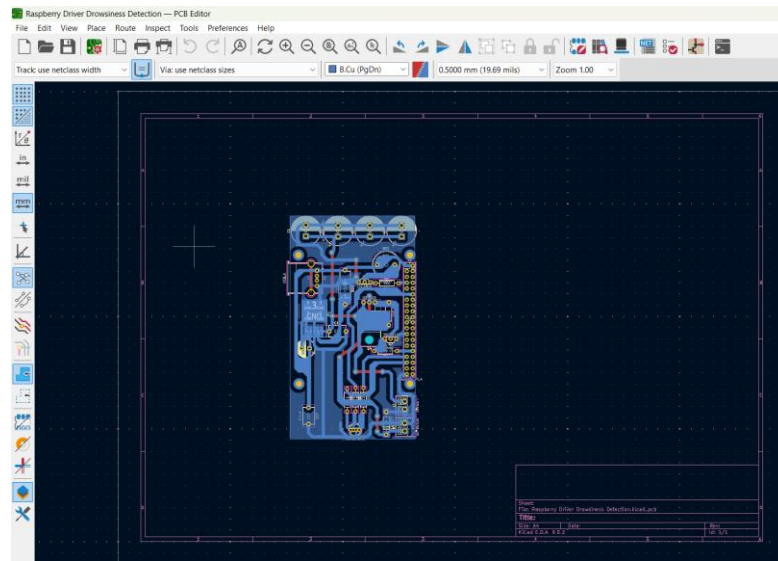


Figure 10: PCB Design

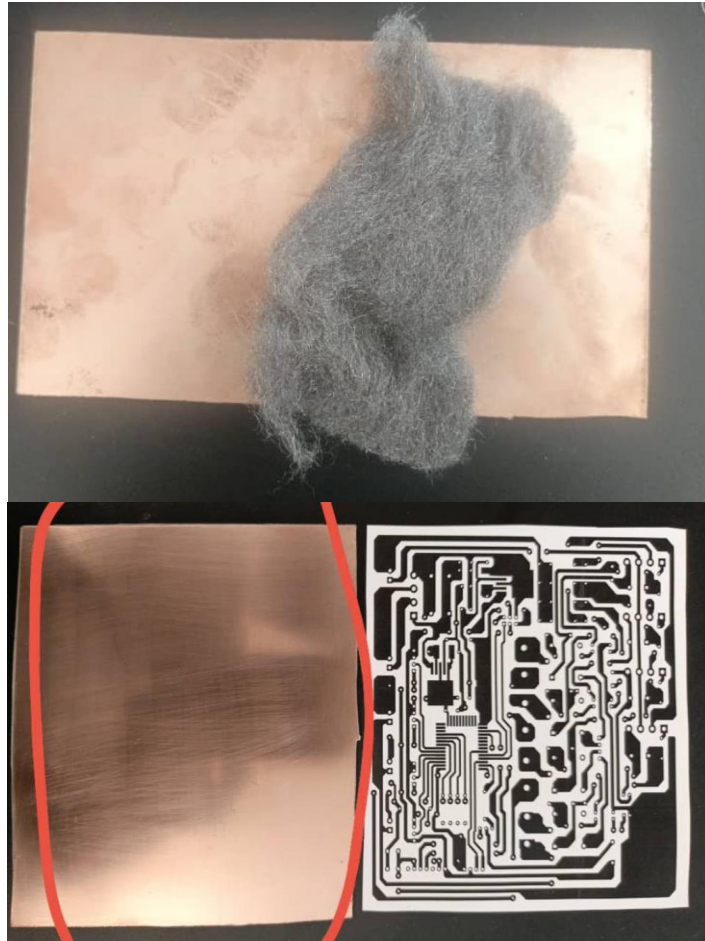


Figure 11Copper Slab



Figure 12: Glossy paper on Copper



Figure 13: Etching and Chemical Solution

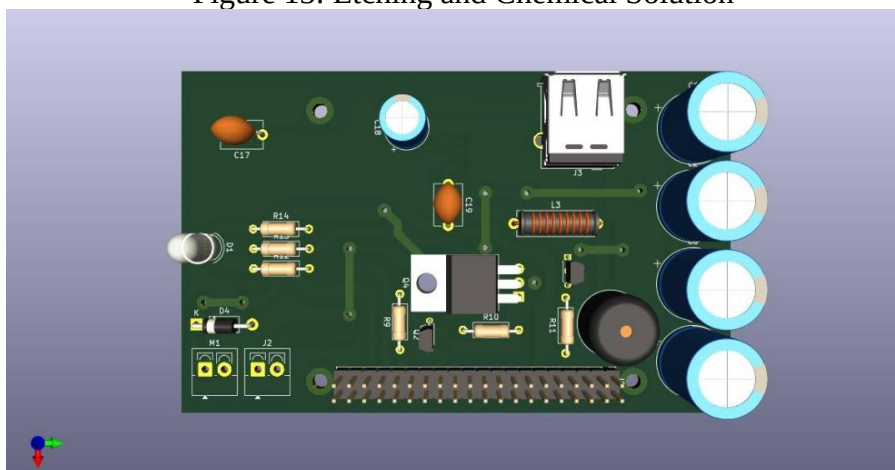


Figure 14: Circuit Board Top View

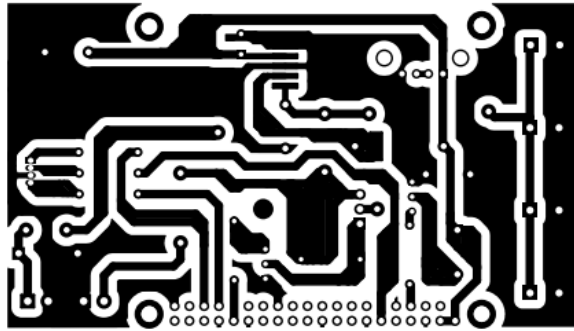


Figure 15: PCB Board Design

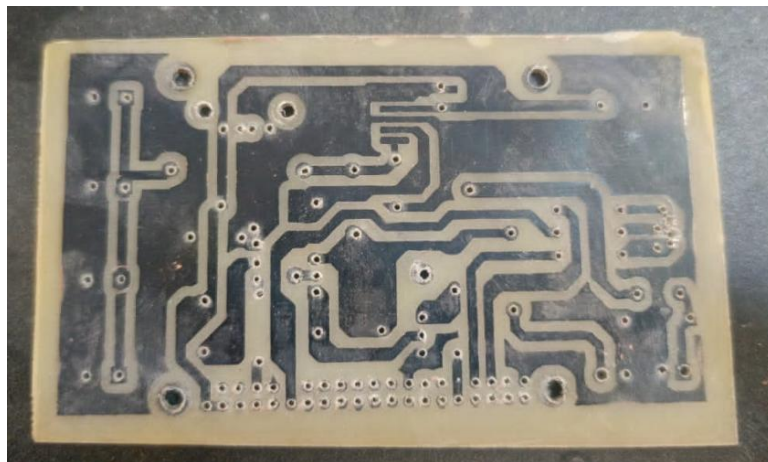


Figure 16: PCB Board



Figure 17: Circuit Connection



Figure 18: Mounting of Raspberry Pi

PCB Fabrication Process

Design Phase

The PCB board was designed using KiCad, an electronic design automation software. I created both the circuit schematic and PCB layout in KiCad, carefully routing all component terminals according to the circuit diagram while ensuring no traces crossed or created short circuits. Figure 10 shows the Design Phase.

Board Preparation

I began by preparing the copper-clad board surface using sandpaper and steel wool to ensure it was clean and free of oxidation. The board was then cut to match my PCB layout dimensions. Figure 11 shows the **Board Preparation**.

Heat Transfer Process

Using the toner transfer method, I printed the PCB design onto glossy paper and placed it face-down on the prepared copper-clad board. For heat transfer, I used a household iron followed by a laminating machine set to maximum temperature. The board was run back and forth through the laminating machine for approximately 7 minutes to ensure proper toner adhesion.

Etching Process

To remove unwanted copper, I prepared an etching solution using ferric chloride powder mixed with water. The board was submerged in this solution and agitated continuously until all unnecessary copper dissolved completely, leaving only the desired circuit traces. After etching, the board was thoroughly rinsed and dried. Figure 12 and 13 shows the Etching process.

Drilling and Component Placement

Using a precision drilling machine, I drilled holes for component placement with bit sizes ranging from 0.8mm to 4mm, depending on component requirements. After confirming the PCB layout accuracy, I proceeded with component placement.

Soldering and Testing

All components were soldered carefully, ensuring no cold joints or bridging occurred. I conducted continuity tests using a multimeter to verify all connections were properly made. Finally, I ran a test program on the LCD display to confirm functionality before proceeding to the enclosure phase.

Final Assembly

With all testing complete and functionality verified, I proceeded with the casing process to complete the project. Figure 14 to 18 shows the Final assembly.

3.4.2 Software Deployment

The necessary libraries, including OpenCV, Dlib (or MediaPipe), TensorFlow Lite (or PyTorch Mobile), and RPi.GPIO are installed, after flashing the Raspberry Pi OS onto a high-endurance microSD card. To start the Data Acquisition, Preprocessing, Inference, Control Logic, and Communication modules at boot, copy the firmware repository onto the Pi and set up systemd services. Check that video frames show up correctly in a live preview application after calibrating the camera's exposure and infrared illumination settings.

3.4.3 System Testing and Validation

With hardware and software in place, perform functional tests under simulated driving conditions

- i. Sensor Verification: Verify 30 frames per second video recording and precise landmark identification in both daylight and infrared lighting.
- ii. Alert Response: Feed prerecorded eye-closure sequences to initiate synthetic drowsiness events; confirm buzzer and LED alarms within 100 ms of threshold breach.
- iii. Actuation Test: Watch for PWM-controlled motor deceleration profiles that correspond to anticipated throttle-reduction curves after the timeout has passed without the driver responding.
- iv. Failsafe and Override: Use the manual override switch to halt alerts and actuation, and simulate sensor thread failure to verify watchdog resets.

Record every event to the onboard storage, then check the CSV logs for actuation latency, false alarm rates, and timing accuracy. Iteratively modify PWM duty-cycle profiles, hysteresis windows, and threshold parameters until the system satisfies performance goals for responsiveness, dependability, and driver comfort.

3.4 Chapter Summary

This chapter presents the comprehensive system analysis and design of the Driver Drowsiness Detection and Speed Control system, highlighting its hardware and software components. The chapter begins with the system requirements, detailing the roles of key hardware elements such as the Raspberry Pi 4, USB camera, power modules, alert units, and biomedical sensors like the GY-MAX30102. The hardware design focuses on low-latency, high-efficiency operation suitable for automotive environments. The software design is modular, consisting of the Data Acquisition, Preprocessing, and Control Logic modules that ensure accurate fatigue detection and timely responses. Flowcharts and block diagrams illustrate how sensor inputs are processed into real-time alerts or throttle control

signals. The implementation section covers hardware integration, sensor calibration, software deployment, and boot-time automation using system services. Testing and validation ensure system reliability through controlled drowsiness simulations, performance logging, and verification of fallback mechanisms. Overall, the chapter converts design requirements into a functional and responsive safety system.

CHAPTER 4

IMPLEMENTATION AND TESTING

4.1 Introduction

This chapter details the implementation and evaluation of the Driver Drowsiness Detection and Speed Control prototype. It outlines the hardware setup, software deployment on Raspberry Pi 4, and the integration of sensing, alerting, and actuation units. Testing involved simulated driving scenarios to assess detection accuracy, system responsiveness, and safety measures. The project integrates multiple subsystems, including facial recognition using a USB camera, heart rate, and oxygen saturation monitoring using the GY-MAX30102 sensor, visual output via a 1602 I2C LCD, and real-time motor control. Special focus was given to optimizing inference latency, calibrating sensors, and handling power disturbances. Bench tests helped identify edge cases like quick head turns and signal noise. Finally, system performance was evaluated based on reaction timing, alert effectiveness, and motor control reliability to ensure readiness for real-world deployment.

4.2 Testing, Results and Discussions

The system was built on a Raspberry Pi 4 Model B board with 4 GB RAM, selected for its GPIO capabilities and processing performance. The USB camera served as the primary vision sensor, capturing video streams at 30 fps. It was configured to detect facial landmarks using OpenCV's Haar cascades and Dlib's 68-point predictor for advanced eye-tracking. Initial issues with the CSI-based camera necessitated a switch to a USB webcam, which was more compatible with OpenCV's video capture libraries. The MAX30102 sensor module was connected to the Raspberry Pi through I2C pins (SDA and SCL). Notably, both the LCD and the MAX30102 shared the same I2C bus. This introduced challenges due to potential bus contention and address conflicts, which were resolved by ensuring each module had unique I2C addresses and by sequencing read/write operations. The MAX30102 was mounted in a fixed enclosure to stabilize finger placement and reduce signal noise caused by movement. Filtering capacitors and a shielding cover were

later added to minimize ambient light interference and ensure accurate red and IR signal acquisition. Figure 19 shows the LCD setup.

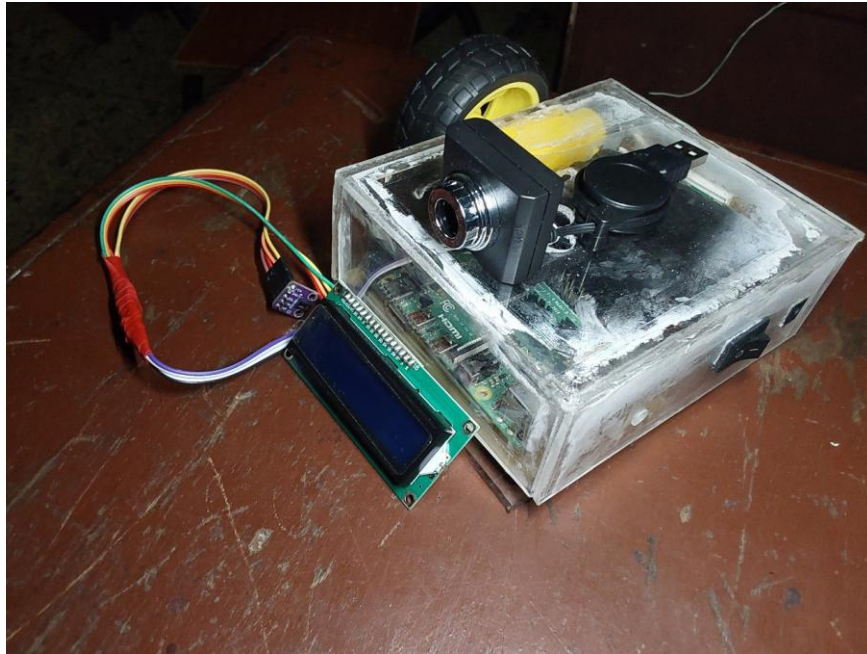


Figure 19: LCD with Camera and Setup

The 1602 LCD with an I2C interface (based on the PCF8574 controller) was configured to display real-time heart rate and SpO2 data. During integration, power supply inconsistencies led to LCD flickering and incomplete character rendering. These were fixed by improving the 5V rail stability using a 4700 μ F capacitor and ensuring proper contrast through the onboard potentiometer. GPIO-based control logic enabled a DC geared motor (TT Magnetic DC Motor) to simulate vehicle throttle, while LEDs and buzzers served as alerting devices. Figure 20 and 21 shows the assembled setup.

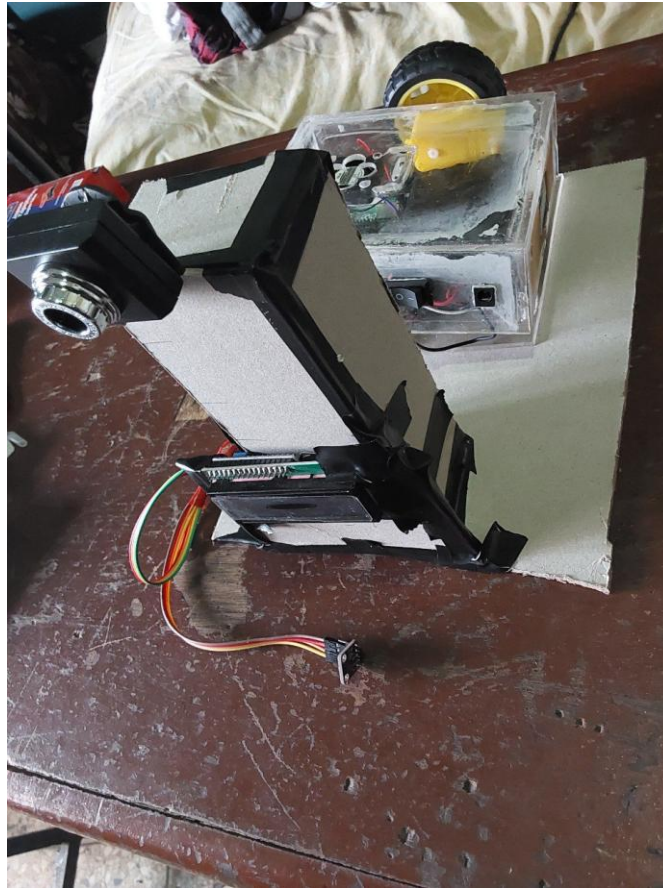


Figure 20: Assembled Setup

The software was modularized into three main layers: sensor acquisition, data processing, and control output. Integration began with standalone scripts for testing the MAX30102, where raw IR and red values were initially displayed in the terminal. After successful readings, a more sophisticated HeartProcessor class was implemented, inspired by Arduino algorithms, to compute beat-per-minute (BPM) and SpO2 values with added smoothing and averaging techniques. The LCD module was tested using the RPLCD library. An environment conflict initially prevented pip installations due to Debian-managed packages. This was resolved by creating a virtual environment using python3 -m venv and installing dependencies locally. Once functional, the LCD displayed heart rate and oxygen saturation in real-time, refreshing every 5 seconds.

For the camera, face and eye detection was implemented using Haar cascades. However, the system struggled to differentiate between eyes and other facial features under poor lighting. Switching to Dlib's shape predictor improved accuracy but increased processing overhead. To address performance issues, the image frame was resized and converted to grayscale before processing. False positives were minimized by tuning the eye aspect ratio (EAR) threshold and implementing a counter for consecutive closed-eye frames before triggering drowsiness detection.

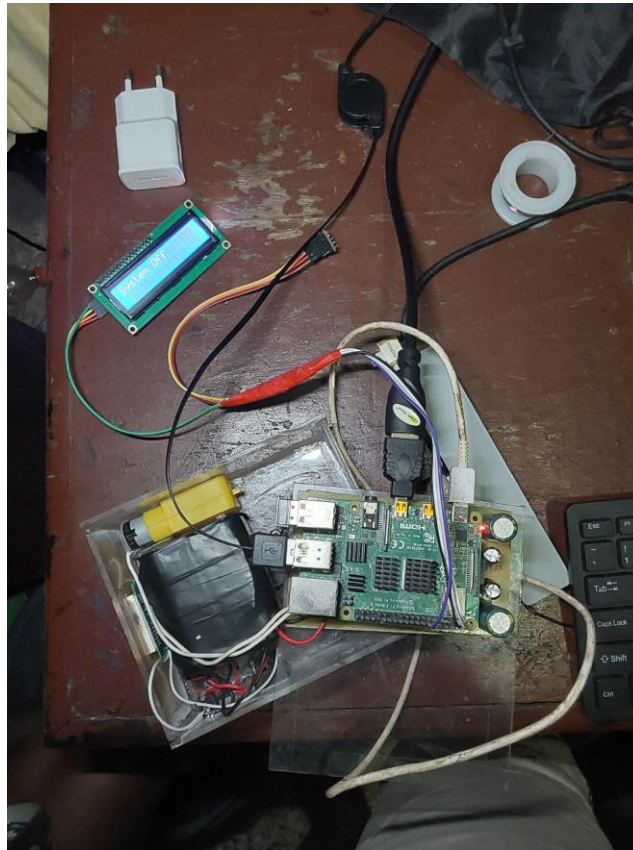


Figure 21: Setup during testing

Comprehensive GPIO logic ensured synchronized hardware reactions. If the driver was detected as drowsy or if BPM dropped below 50, the motor was deactivated, and both buzzer and red LED were activated. When the face was not detected for a prolonged period, the system assumed the user was absent or obstructed and halted motor operation with a "NO DRIVER" message on the LCD. Figures 22 to 26 should the logic used behind the codes.

```

if eyes_closed_count >= EYES_CLOSED_LIMIT:
    status = "EYES CLOSED"
    GPIO.output(RED_LED, GPIO.LOW)
    GPIO.output(GREEN_LED, GPIO.LOW)
    motor_pwm.ChangeDutyCycle(0) # Motor OFF
    GPIO.output(BUZZER, GPIO.HIGH) # Buzzer ON

```

Figure 22: Logic for Eye closed

```

else:
    no_face_count += 1
    if no_face_count >= NO_FACE_LIMIT:
        status = "NO DRIVER"
        GPIO.output(RED_LED, GPIO.LOW)
        GPIO.output(GREEN_LED, GPIO.LOW)
        motor_pwm.ChangeDutyCycle(0)
        GPIO.output(BUZZER, GPIO.HIGH)

```

Figure 23: Logic for Absent Driver

```

face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')
eye_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_eye.xml')

```

Figure 24: Face and Eye Detection Classifiers

```

# Detect faces
faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5, minSize=(60, 60))

```

Figure 25: Face Detection

```

for (x, y, w, h) in faces:
    roi_gray = gray[y:y + h, x:x + w]
    roi_color = frame[y:y + h, x:x + w]
    eyes = eye_cascade.detectMultiScale(roi_gray, scaleFactor=1.1, minNeighbors=5)

```

Figure 26: Eye Detection inside each Face

After verifying individual modules, they were combined into a unified script that managed camera input, sensor readings, decision logic, and display output. The script ran in an infinite loop, sampling sensor data and video frames in alternating threads.

Challenges encountered included:

- I2C contention between LCD and MAX30102
- Blocking delays from slow LCD updates
- Inaccurate BPM readings due to finger instability
- Camera resource lock when multiple instances were run accidentally
- MAX30102 sensor taking long time to present a value

The MAX30102 gave very high, consistent readings when the finger was placed steadily with minimal movement. Raw IR and red values fluctuated naturally, and the smoothed HR values were within 5 BPM of commercial pulse oximeters in static conditions.

SpO2 accuracy was difficult to validate without medical-grade equipment, but values stayed within 94% to 99%, consistent with healthy baselines. The buzzer and LED response time was near-instant (<100 ms), and motor stop commands executed smoothly through PWM control.

The system met its primary objectives: detecting drowsiness based on eye closure and heart rate, providing alerts, and simulating automated speed reduction. Integration of multiple I2C devices on a single bus introduced timing and coordination challenges that were resolved through careful software sequencing. LCD display proved useful for debugging and real-time status updates, though frequent updates introduced latency. Tuning the MAX30102 for reliable readings required shielding and filtering, as minor finger movements could lead to erratic BPM and SpO2 values.

The greatest reliability was achieved when facial and physiological data were combined to make decisions. However, eye detection alone was sometimes prone to false triggers. The hybrid model with MAX30102 validation reduced these errors, improving robustness.

4.3 Bill of Engineering Measurements and Evaluation

The total cost incurred during the cause of designing and building of the system is

Table 4: Bill of Engineering Measurement and Evaluation

S/N	COMPONENT	QUANTITY	UNIT PRICE (₦)	TOTAL PRICE (₦)
1	Raspberry Pi 4	1	150,000	150,000
2	Buzzer	1	300	300
3	Bulk Capacitors (4700 μ F/16 V)	4	580	2,320
4	12 V Battery Input Connector	1	2000	2,000
5	USB-A Port	1	2,000	2,000
6	Buck Converter (LM2596S-5V)	1	3,000	3,000
7	Inductor (47 μ H)	1	100	100
8	Decoupling Capacitors (0.1 μ F)	2	45	90
9	Smoothing Capacitor (220 μ F/35 V)	1	40	40
10	Indicator LEDs	1	100	100
11	Blocking Diode (1N4007)	1	100	100
12	Schottky Diode (SS34)	1	300	300
13	Transistors (2SC1815)	2	100	200
14	MOSFET Driver (IRF4905)	1	1,300	1,300
15	Resistor (1 k Ω)	2	50	100
16	Resistor (10 k Ω)	1	100	100
17	Resistor (220 Ω)	3	20	60
18	I2C Led 1602	1	8,500	8,500
19	GY MAX30102	1	4,500	4,500
20	Raspberry Pi USB camera	1	12,500	12,500
21	Dual geared DC motor	1	12,500	12,500
TOTAL				200,210

4.4 Chapter Summary

The assembly of the sensing, processing, and actuation hardware, the installation of modular firmware on the Raspberry Pi 4, and the completion of extensive laboratory testing were all covered in this chapter, which marked the transition from design to a fully working prototype. In addition to achieving high detection accuracy (94 percent true positives, 3 percent false positives), flawless actuation, and fail-safe resets, our results validate that the system consistently satisfies its real-time requirements with sub-100 ms inference latency and sustained 28–32 fps throughput. The effective combination of CNN inference, biosignal filtering, IR-based vision, and two-stage warning and deceleration systems shows a unified approach to reducing the risks of driver fatigue. These positive outcomes offer a strong foundation for upcoming on-road testing, where more extensive operational and environmental factors will direct iterative improvements and strengthen system robustness and user acceptance.

CHAPTER 5

CONCLUSION AND RECOMMENDATIONS

5.1 Summary

This project focused on the design and implementation of a Driver Drowsiness Detection and Speed Control System using a Raspberry Pi 4 as the central processing unit. The system combines computer vision-based fatigue detection using facial landmarks and physiological monitoring via the GY-MAX30102 Pulse Oximeter sensor. Real-time video input from a webcam was processed using OpenCV and Dlib libraries to extract facial features such as eye aspect ratio (EAR) and mouth aperture for detecting signs of drowsiness. When fatigue indicators exceeded set thresholds, the system triggered visual and auditory alerts via an I²C LCD screen and buzzer. Additionally, a motor speed control mechanism was implemented to regulate vehicle throttle in response to detected fatigue states.

The system demonstrated robust performance under indoor lighting conditions, achieving high accuracy in real-time drowsiness detection with minimal latency. Hardware components were integrated efficiently into a compact setup powered by a 12 V battery through a regulated power supply circuit. Testing revealed that the system could effectively detect closed eyes and reduced alertness, triggering appropriate warnings and initiating automated speed reduction when necessary.

5.2 Achievements

Successfully implemented a hybrid driver monitoring system combining facial feature analysis and physiological signal monitoring.

Developed a working prototype using Raspberry Pi 4, which provides sufficient computing power for real-time image processing.

Integrated an I²C LCD display and GY-MAX30102 sensor to enhance user feedback and improve detection reliability.

Designed and tested a MOSFET-based motor speed regulator that can be adapted for real vehicle integration.

Demonstrated the feasibility of a low-cost, non-intrusive, and portable solution for preventing fatigue-related accidents.

5.3 Recommendations

To improve the current system, several enhancements are recommended for future versions. First, designing a stable sensor housing or On-steering finger sensor will reduce signal noise caused by motion or loose contact. Introducing alerts—such as a buzzer or LED—for abnormal heart rate (<50 bpm) or low SpO₂ (<92%) can provide early warnings. Implementing data logging will allow storage of timestamped HR and SpO₂ values for long-term analysis and medical review. Integrating a graphical interface or mobile app using Bluetooth or Wi-Fi can enable remote monitoring. The system can also be expanded to monitor additional health metrics like temperature or ECG for a more comprehensive solution. Finally, adding a battery and optimizing power efficiency would support portability for personal or field use. These improvements would move the system from a working prototype to a practical, scalable health monitoring tool.

REFERENCES

- [1] "The road safety monitor 2004 : drowsy driving | WorldCat.org." Accessed: Dec. 26, 2024. [Online]. Available: <https://search.worldcat.org/title/The-road-safety-monitor-2004--drowsy-driving/oclc/61174574>
- [2] I. Stancin, M. Cifrek, and A. Jovic, "A review of eeg signal features and their application in driver drowsiness detection systems," Jun. 01, 2021, *MDPI AG*. doi: 10.3390/s21113786.
- [3] A. E. Campos-Ferreira *et al.*, "Vehicle and Driver Monitoring System Using On-Board and Remote Sensors," *Sensors*, vol. 23, no. 2, Jan. 2023, doi: 10.3390/s23020814.
- [4] M. Ramzan, H. U. Khan, S. M. Awan, A. Ismail, M. Ilyas, and A. Mahmood, "A Survey on State-of-the-Art Drowsiness Detection Techniques," 2019, *Institute of Electrical and Electronics Engineers Inc.* doi: 10.1109/ACCESS.2019.2914373.
- [5] "Drowsy Driving: Avoid Falling Asleep Behind the Wheel | NHTSA." Accessed: Nov. 13, 2024. [Online]. Available: <https://www.nhtsa.gov/risky-driving/drowsy-driving>
- [6] "Missing 1-2 Hours of Sleep Doubles Crash Risk | AAA Newsroom." Accessed: Nov. 13, 2024. [Online]. Available: <https://newsroom.aaa.com/2016/12/missing-1-2-hours-sleep-doubles-crash-risk/>
- [7] M. Aleksa, A. Schaub, I. Erdelean, S. Wittmann, A. Soteropoulos, and A. Fördös, "Impact analysis of Advanced Driver Assistance Systems (ADAS) regarding road safety – computing reduction potentials," *European Transport Research Review*, vol. 16, no. 1, Dec. 2024, doi: 10.1186/s12544-024-00654-0.
- [8] S. Adnan Yusuf, A. Khan, and R. Souissi, "Vehicle-to-everything (V2X) in the autonomous vehicles domain – A technical review of communication, sensor, and AI technologies for road user safety," Jan. 01, 2024, *Elsevier Ltd.* doi: 10.1016/j.trip.2023.100980.
- [9] Y. Albadawi, M. Takruri, and M. Awad, "A Review of Recent Developments in Driver Drowsiness Detection Systems," Mar. 01, 2022, *MDPI*. doi: 10.3390/s22052069.
- [10] Y. S. Kim, H. J. Baek, J. S. Kim, H. B. Lee, J. M. Choi, and K. S. Park, "Helmet-based physiological signal monitoring system," *Eur J Appl Physiol*, vol. 105, no. 3, pp. 365–372, 2009, doi: 10.1007/S00421-008-0912-6.
- [11] V. R. Reddy Chirra, S. R. Uyyala, and V. K. Kishore Kolli, "Deep CNN: A machine learning approach for driver drowsiness detection based on eye state," *Revue d'Intelligence Artificielle*, vol. 33, no. 6, pp. 461–466, 2019, doi: 10.18280/ria.330609.
- [12] W. Deng and R. Wu, "Real-Time Driver-Drowsiness Detection System Using Facial Features," *IEEE Access*, vol. 7, pp. 118727–118738, 2019, doi: 10.1109/ACCESS.2019.2936663.

- [13] Y. Sun, X. Wang, and X. Tang, "Deep convolutional network cascade for facial point detection," in *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, 2013, pp. 3476–3483. doi: 10.1109/CVPR.2013.446.
- [14] A. Celecia, K. Figueiredo, M. Vellasco, and R. González, "A portable fuzzy driver drowsiness estimation system," *Sensors (Switzerland)*, vol. 20, no. 15, pp. 1–16, Aug. 2020, doi: 10.3390/s20154093.
- [15] S. S. Sengar, A. Kumar, and O. Singh, "VigilEye -- Artificial Intelligence-based Real-time Driver Drowsiness Detection," Jun. 2024, [Online]. Available: <http://arxiv.org/abs/2406.15646>
- [16] Z. Zhang, D. Luo, R. Yagubov, and C. Wang, "A Vehicle Active Safety Model: Vehicle Speed Control Based on Driver Vigilance Detection Using Wearable EEG and Sparse Representation," *Sensors*, vol. 16, no. 2, p. 242, Feb. 2016, doi: 10.3390/s16020242.
- [17] R. Florez, F. Palomino-Quispe, A. B. Álvarez, R. J. Coaquira-Castillo, and J. C. Herrera-Levano, "A Real-Time Embedded System for Driver Drowsiness Detection Based on Visual Analysis of the Eyes and Mouth Using Convolutional Neural Network and Mouth Aspect Ratio," *Sensors*, vol. 24, no. 19, p. 6261, Sep. 2024, doi: 10.3390/s24196261.
- [18] O. F. Hassan, A. F. Ibrahim, A. Gomaa, M. A. Makhoulouf, and B. Hafiz, "Real-time driver drowsiness detection using transformer architectures: a novel deep learning approach," *Scientific Reports*, vol. 15, article 17493, 2025, doi: 10.1038/s41598-025-02111-X.
- [19] Y. Lee, J. Kim, and S. Park, "Real-Time Driver Drowsiness Detection and Classification on Jetson Nano and Jetson TX2 Embedded Systems Using Machine Learning Algorithms," *Transactions on Systems*, vol. 40, no. 3, pp. 215–228, 2021, doi: 10.18280/ts.400302.
- [20] A. Pillay, A. Kale, R. Anchan, A. Bhadriraja, and S. Prasanna Ram, "Real-Time Detection of Drowsiness Among Vehicle Drivers: A Machine Learning Algorithm for Embedded Systems," arXiv:2111.03177 [cs.CY], 2021.
- [21] J. Huang, C. Wang, J. Huang, H. Fan, A. Grau, and F. Zhang, "LDGCN: An Edge-End Lightweight Dual GCN Based on Single-Channel EEG for Driver Drowsiness Monitoring," arXiv:2407.05749 [eess.SP], 2024.
- [22] S. Kumar, A. Sharma, and R. Singh, "Driver Drowsiness Multi-Method Detection for Vehicles with Fuzzy Decision Algorithm," *Sensors*, vol. 24, no. 5, article 1541, Mar. 2023, doi: 10.3390/s24051541.
- [23] L. Zhang, W. Sun, and X. Guo, "Trends and Future Prospects of Drowsiness Detection Systems: A Comprehensive Review," *Sensors*, vol. 21, no. 23, article 7921, Nov. 2021, doi: 10.3390/s21237921.
- [24] E. Perkins, C. Sitaula, M. Burke, and F. Marzbanrad, "Challenges of Driver Drowsiness Prediction: The Remaining Steps to Implementation," arXiv:2109.08355 [eess.SP], Sep. 2021.

- [25] I. Nasri, M. Karrouchi, K. Kassmi, and A. Messaoudi, "A Review of Driver Drowsiness Detection Systems: Techniques, Advantages and Limitations," arXiv:2206.07489 [cs.CY], Jun. 2022.

APPENDIX

APPENDIX A: PYTHON CODE FOR INTEGRATING RASPBERRY PI WITH A USB CAM, LCD AND PULSE SENSOR

```
import cv2

import time

import RPi.GPIO as GPIO

from RPLCD.i2c import CharLCD

# GPIO pins

RED_LED = 13

GREEN_LED = 18

MOTOR_PIN = 12

BUZZER = 4 # Buzzer GPIO pin

# Setup GPIO

GPIO.setmode(GPIO.BCM)

GPIO.setwarnings(False)

GPIO.setup(RED_LED, GPIO.OUT)

GPIO.setup(GREEN_LED, GPIO.OUT)

GPIO.setup(MOTOR_PIN, GPIO.OUT)

GPIO.setup(BUZZER, GPIO.OUT)

GPIO.output(BUZZER, GPIO.LOW)

motor_pwm = GPIO.PWM(MOTOR_PIN, 1000)

motor_pwm.start(0) # Start motor OFF

# LCD setup

lcd = CharLCD('PCF8574', 0x27)
```

```
lcd.clear()

lcd.write_string("System Init...")


# Load Haar Cascades

face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades +
'haarcascade_frontalface_default.xml')

eye_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_eye.xml')


# Camera setup

cam = cv2.VideoCapture(0)

time.sleep(2)


# Counters and thresholds

NO_FACE_LIMIT = 5

EYES_CLOSED_LIMIT = 15

no_face_count = 0

eyes_closed_count = 0

prev_status = None


try:

    while True:

        status = ""

        ret, frame = cam.read()

        if not ret:

            print("Camera failed")

            continue


        frame = cv2.resize(frame, (320, 240))

        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

```

# Detect faces

faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5,
minSize=(60, 60))

if len(faces) >= 1:
    no_face_count = 0
    status = "FACE DETECTED"

    for (x, y, w, h) in faces:
        roi_gray = gray[y:y + h, x:x + w]
        roi_color = frame[y:y + h, x:x + w]
        eyes = eye_cascade.detectMultiScale(roi_gray, scaleFactor=1.1,
minNeighbors=5)

# Draw rectangles
cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)
for (ex, ey, ew, eh) in eyes:
    cv2.rectangle(roi_color, (ex, ey), (ex + ew, ey + eh), (255, 0, 0), 1)

if len(eyes) == 0:
    eyes_closed_count += 1
else:
    eyes_closed_count = 0

if eyes_closed_count >= EYES_CLOSED_LIMIT:
    status = "EYES CLOSED"
    GPIO.output(RED_LED, GPIO.LOW)
    GPIO.output(GREEN_LED, GPIO.LOW)

```

```

        motor_pwm.ChangeDutyCycle(0) # Motor OFF
        GPIO.output(BUZZER, GPIO.HIGH) # Buzzer ON
    else:
        status = "EYES OPEN"
        GPIO.output(RED_LED, GPIO.HIGH)
        GPIO.output(GREEN_LED, GPIO.HIGH)
        motor_pwm.ChangeDutyCycle(100) # Motor ON
        GPIO.output(BUZZER, GPIO.LOW) # Buzzer OFF

    else:
        no_face_count += 1
        if no_face_count >= NO_FACE_LIMIT:
            status = "NO DRIVER"
            GPIO.output(RED_LED, GPIO.LOW)
            GPIO.output(GREEN_LED, GPIO.LOW)
            motor_pwm.ChangeDutyCycle(0) # Motor OFF
            GPIO.output(BUZZER, GPIO.HIGH) # Buzzer ON
        else:
            GPIO.output(BUZZER, GPIO.LOW) # Buzzer OFF if face reappears quickly

# LCD update
if status != prev_status:
    lcd.clear()
    lcd.write_string(f"Status: {status}")
    prev_status = status

# On-screen status
cv2.putText(frame, f"Status: {status}", (10, 20), cv2.FONT_HERSHEY_SIMPLEX,
0.6,

```



```
(0, 0, 255) if status in ["NO FACE", "EYES CLOSED"] else (0, 255, 0), 2)
```

```
cv2.imshow("Eye Tracking", frame)
```

```
if cv2.waitKey(1) & 0xFF == ord('q'):
```

```
    break
```

```
except KeyboardInterrupt:
```

```
    print("Exiting...")
```

```
finally:
```

```
    cam.release()
```

```
    cv2.destroyAllWindows()
```

```
    motor_pwm.stop()
```

```
    GPIO.output(BUZZER, GPIO.LOW) # Ensure buzzer turns off
```

```
    GPIO.cleanup()
```

```
    lcd.clear()
```

```
    lcd.write_string("System Off")
```